รายงานโครงงาน **Intro ML**

บทที่ **1** บทนำ

ฐานข้อมูลลูกค้าชุดเดียวใช้ตอบการตัดสินใจสองข้อ คือ ควรเสนอแคมเปญให้ลูกค้ารายใด (Response) และ ลูกค้ารายนั้นมีรายได้ต่อปีเท่าใด (Income) โครงงานนี้สร้างและเปรียบเทียบ Model ตามลำดับเวลาของการพัฒนา ตั้งแต่โมเดลพื้นฐานจนถึงโมเดลนอกเหนือจากที่เรียนในห้อง

1.1 ที่มาและความสำคัญ

- บริษัทขายปลีกมีลูกค้าจำนวนมาก แต่มีเพียง 14.9% ที่ตอบรับแคมเปญล่าสุด การจัดอันดับลูกค้าที่มีโอกาสตอบรับสูงช่วยลดต้นทุนการส่งข้อเสนอ
- Income เป็นข้อมูลอ่อนไหวและมักขาดหาย (ในชุดข้อมูลนี้ขาด 24 แถว) จึงต้องประมาณจากพฤติกรรมการซื้อและช่องทางที่ลูกค้าใช้
- ชุดข้อมูลเดิมสร้างมาเพื่อแบ่งกลุ่มลูกค้า (clustering) เรานำมาใช้ใหม่เพื่อการพยากรณ์แบบมีผู้สอน (supervised) สองงาน

1.2 วัตถุประสงค์

1. สร้างโมเดล classification Response
2. สร้างโมเดล regression Income
3. ทุกโมเดลมีสองแบบ คือ Library และ From Scratch
4. สร้างโมเดลใหม่นอกเหนือจากที่เรียนอย่างน้อยงานละหนึ่งโมเดล และอธิบายกลไกภายใน
5. ตรวจสอบความทนทานของผลด้วย k-fold cross-validation
6. ตอบคำถาม 3 ข้อจากอาจารย์หลังนำเสนอ: Feature interaction, ทำไม FT-Transformer ใช้ ReGLU และ extracurricular ชนะโมเดลในหลักสูตรจริงหรือไม่

1.3 ชุดข้อมูล

รายการ	รายละเอียด
แหล่งที่มา	Kaggle imakash3011/customer-personality-analysis (CC0)
ขนาด	2,240 แถว × 29 คอลัมน์ คั่นด้วย tab
กลุ่มคอลัมน์	People, Products, Promotion, Place
classification target	Response (14.94% ตอบรับ / 85.06% ไม่ตอบรับ)
regression target	Income (เฉลี่ย $51,959 มัธยฐาน $51,371)
cleaned	2,236 แถว (clf) / 2,212 แถว (reg)

ตารางที่ **1.1** รายละเอียดของชุดข้อมูล

1.4 ขอบเขตและตัวชี้วัด

classify ใช้ **F1** เป็น Main Metric เพราะข้อมูลไม่สมดุล (Data Imbalance: ตอบรับเพียง 14.9%) การตอบ "ไม่ตอบรับ" ทุกแถวก็ได้ accuracy 0.851 (85%) แล้ว ทั้งที่ไม่ได้ลูกค้าที่ตอบรับเลยสักคน accuracy จึงใช้ตัดสินไม่ได้ F1 รวม precision และ recall ไว้ในค่าเดียว จะสูงได้ก็ต่อเมื่อดีทั้งสองด้าน ส่วน regression ใช้ **R^2** เป็น Main Metric เพราะเป็นตัววัดทั่วไปที่ไม่ขึ้นกับหน่วย และโดยทั่วไปอยู่ในช่วง 0–1 (1 = ทำนายได้สมบูรณ์) จึงประเมินง่าย และรายงาน RMSE (หน่วยดอลลาร์) เป็น Sub Metrics

$$F1 = \frac{2 \cdot P \cdot R}{P + R}, \qquad P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

ตัวชี้วัดอื่นที่รายงานเพิ่ม: BCE loss, accuracy, precision, recall (sensitivity), ROC-AUC สำหรับ classify และ MSE, RMSE, MAE สำหรับ regression

1.5 โครงสร้างรายงาน

- บทที่ 2 การวิเคราะห์ข้อมูลเชิงสำรวจ (EDA) การคัดเลือกฟีเจอร์ และ Feature Interaction

- บทที่ 3 ระเบียบวิธีและการตั้งค่าการทดลอง (รวมกติการอบที่สอง)
- บทที่ 4 ลำดับการพัฒนาโมเดลและเหตุผลเบื้องหลัง (V1.0 ถึง V7)
- บทที่ 5 ผลเปรียบเทียบทุกโมเดล คำตอบว่า extracurricular ชนะหรือไม่ และการอภิปราย
- บทที่ 6 สรุปและข้อเสนอแนะ

บทที่ **2** การวิเคราะห์ข้อมูลเชิงสำรวจ **(EDA)**

จาก 29 คอลัมน์เหลือ 25 ตัวแปร ใช้การโหวต 3 วิธีเลือกฟีเจอร์เหลือ **18** ตัว **(22** หลัง **one-hot)** สำหรับ **Response** และ **13** ตัว สำหรับ **Income** บทนี้สรุปผล EDA

2.1 ความหมายของ **Feature**

กลุ่ม	Feature	ความหมาย
People	ID	รหัสประจำตัวลูกค้า
People	Year_Birth	ปีเกิดของลูกค้า
People	Education	ระดับการศึกษา
People	Marital_Status	สถานะสมรส
People	Income	รายได้ต่อปีของครัวเรือน (ดอลลาร์)
People	Kidhome	จำนวนเด็กเล็กในบ้าน
People	Teenhome	จำนวนวัยรุ่นในบ้าน
People	Dt_Customer	วันที่สมัครเป็นลูกค้า
People	Recency	จำนวนวันนับจากการซื้อครั้งล่าสุด
People	Complain	1 = เคยร้องเรียนใน 2 ปีที่ผ่านมา, 0 = ไม่เคย
Products	MntWines	ยอดซื้อไวน์ใน 2 ปีที่ผ่านมา
Products	MntFruits	ยอดซื้อผลไม้ใน 2 ปีที่ผ่านมา
Products	MntMeatProducts	ยอดซื้อเนื้อสัตว์ใน 2 ปีที่ผ่านมา

กลุ่ม	Feature	ความหมาย
Products	MntFishProducts	ยอดซื้อปลาใน 2 ปีที่ผ่านมา
Products	MntSweetProducts	ยอดซื้อขนมหวานใน 2 ปีที่ผ่านมา
Products	MntGoldProds	ยอดซื้อสินค้าทองคำใน 2 ปีที่ผ่านมา
Promotion	NumDealsPurchases	จำนวนครั้งที่ซื้อด้วยส่วนลด
Promotion	AcceptedCmp1	1 = ตอบรับข้อเสนอแคมเปญที่ 1, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp2	1 = ตอบรับข้อเสนอแคมเปญที่ 2, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp3	1 = ตอบรับข้อเสนอแคมเปญที่ 3, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp4	1 = ตอบรับข้อเสนอแคมเปญที่ 4, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp5	1 = ตอบรับข้อเสนอแคมเปญที่ 5, 0 = ไม่ตอบรับ
Promotion	Response	1 = ตอบรับข้อเสนอแคมเปญล่าสุด, 0 = ไม่ตอบรับ (เป้าหมาย classify)
Place	NumWebPurchases	จำนวนครั้งที่ซื้อผ่านเว็บไซต์
Place	NumCatalogPurchases	จำนวนครั้งที่ซื้อผ่านแคตตาล็อก
Place	NumStorePurchases	จำนวนครั้งที่ซื้อที่หน้าร้าน
Place	NumWebVisitsMonth	จำนวนครั้งที่เข้าเว็บไซต์ในเดือนที่ผ่านมา
ค่าคงที่	Z_CostContact	เท่ากับ 3 ทุกแถว ไม่ระบุความหมาย (ลบทิ้ง)

กลุ่ม	Feature	ความหมาย
ค่าคงที่	Z_Revenue	เท่ากับ 11 ทุกแถว ไม่ระบุความหมาย (ลบทิ้ง)
สร้างเพิ่ม	Age	อายุ = 2014 – Year_Birth
สร้างเพิ่ม	Customer_Tenure_Days	จำนวนวันจากวันสมัครถึงวันสมัครล่าสุดในข้อมูล (2014-06-29)

ตารางที่ **2.1** คำอธิบายฟีเจอร์แต่ละกลุ่ม

2.2 การ **Clean Data**

ทำการ clean data ตาม 7 ขั้นตอนดังต่อไปนี้

ลำดับ	การดำเนินการ	เหตุผล / ผล
1	ลบ ID	เป็นรหัสประจำตัวไม่ใช่ฟีเจอร์
2	ลบ Z_CostContact, Z_Revenue	ค่าคงที่ ไม่มีความแปรปรวน
3	แปลง Dt_Customer เป็น Customer_Tenure_Days	จำนวนวันจากวันสมัครถึงวันสมัครล่าสุด (2014-06-29) ช่วง 0–699 วัน เฉลี่ย 353.6 วัน
4	สร้าง Age = 2014 – Year_Birth และลบ 3 แถวที่ปีเกิดผิดปกติ (1893, 1899, 1900)	อายุ 121, 115, 114 ปี เป็นข้อผิดพลาด การกรอก แถวเหลือ 2,237
5	รวม Absurd, YOLO, Alone เข้ากับ Single	สองค่าแรกเป็นคำตอบล้อเล่น Alone ความหมายเท่า Single และมีจำนวนน้อย Single เพิ่มจาก 479 เป็น 486 เหลือ 5 หมวด
6	Income หาย 24 แถว	classify: เก็บแถวไว้และเติมค่ามัธยฐานในฐานะฟีเจอร์ regression: ลบ 24 แถวนั้น เพราะ Income คือเป้าหมาย
7	ลบแถวลูกค้า 9432 ที่ Income = $666,666	ค่าผิดปกติสุดขีด รายได้สูงสุดหลังลบเหลือ $162,397

ตารางที่ **2.2** ขั้นตอนการทำความสะอาดข้อมูล

2.3 การกระจายตัวของตัวแปรเป้าหมายและการแบ่งชุดข้อมูล

แบ่งชุดข้อมูล 80/20 ด้วย `random_state = 42` (stratified)

ชุด	n	Yes (1)	No (0)
Full (classify)	2,236	334 (14.94%)	1,902 (85.06%)
Train	1,788	267 (14.93%)	1,521 (85.07%)
Test	448	67 (14.96%)	381 (85.04%)

ตารางที่ **2.3** การกระจายของ `Response` ในแต่ละชุดข้อมูล

Income	Full (2,212)	Test (n = 443)
ค่าเฉลี่ย	$51,959	$53,038
มัธยฐาน	$51,371	$51,948
ส่วนเบี่ยงเบนมาตรฐาน	$21,527	$21,819
ต่ำสุด / สูงสุด	$1,730 / $162,397	$4,023 / $162,397

ตารางที่ **2.4** ค่าสถิติของ `Income`

2.4 ความสัมพันธ์กับตัวแปรเป้าหมาย **(relevance)**

ใช้สองตัววัดตามชนิดฟีเจอร์: ตัวเลขใช้ point-biserial (classify) หรือ Pearson (regression) หมวดใช้ Cramér's V หรืออัตราส่วนสหสัมพันธ์ η เกณฑ์ผ่านคือ 0.10 ทั้ง `Response` และ `Income` มีสัมพันธ์ชัดกว่าชุดข้อมูลอื่นที่เคยทำมา

classify `Response`: 19 จาก **25** ตัวผ่าน **0.10**

- สูงสุดคือ `AcceptedCmp5` (0.328), `AcceptedCmp1` (0.294), `AcceptedCmp3` (0.254), `MntWines` (0.247) และ `MntMeatProducts` (0.237)
- `AcceptedCmp1–5` ไม่ใช่ **data leakage**: เป็นประวัติคามเปญในอดีต มีอยู่ก่อนแคมเปญของ `Response` เสมอ อัตราตอบรับเท่ากับ **40.7%** (n = 462) หากเคย

ตอบรับแคมเปญใดแคมเปญหนึ่ง เทียบกับ **8.2%** (n = 1,775) หากไม่เคย ข้อจำกัดคือลูกค้าใหม่ที่ไม่มีประวัติจะไม่มีฟีเจอร์กลุ่มนี้ (cold-start)

- `Recency` (0.199) และ `Customer_Tenure_Days` (0.194) สอดคล้องกับหลัก RFM: ซื้อล่าสุดใกล้กว่าและอยู่กับบริษัทนานกว่า ตอบรับมากกว่า
- `Teenhome` (0.155) ผ่าน แต่ `Kidhome` (0.080) ไม่ผ่าน บ้านที่มี `Teenhome` ตอบรับมากกว่าบ้านที่มี `Kidhome`
- ไม่ผ่าน: `Kidhome`, `NumStorePurchases` (0.039), `Age` (0.018), `NumWebVisitsMonth` (0.004), `NumDealsPurchases` (0.002), `Complain` (0.0002)

regression `Income`: 17 จาก **25** ตัวผ่าน **0.10** ซึ่งแข็งแกร่งกว่า **classify** มาก

- สูงสุดคือ `NumCatalogPurchases` (0.589), `MntMeatProducts` (0.584), `MntWines` (0.578) และ `NumWebVisitsMonth` (−0.553)
- `NumWebVisitsMonth` เป็นความสัมพันธ์เชิงลบ ผู้ที่มี `NumWebVisitsMonth` สูง มีรายได้น้อยกว่า และ `Kidhome` (−0.428) บ้านที่มี `Kidhome` สูงมีรายได้เฉลี่ยต่ำกว่า
- `Education` (η = 0.218): ระดับ `Basic` มีรายได้เฉลี่ย $20,306 ต่ำกว่าระดับอื่นประมาณ 2.5 เท่า (ระดับอื่นอยู่ในช่วง $47.6k–$56.1k)
- ผู้ตอบรับแคมเปญมีรายได้มัธยฐาน $64,090 สูงกว่าผู้ไม่ตอบรับ ($50,150) สูงกว่า **27.8%** ซึ่งเป็นความสัมพันธ์เดียวกันกับ `Income` ใน classify (0.133) มองจากอีกด้าน

2.5 ความสัมพันธ์ร่วมระหว่างฟีเจอร์ **(multicollinearity)**

คำนวณที่ Pearson ระหว่างฟีเจอร์เชิงตัวเลข 23 ตัว (ฟีเจอร์กลุ่มใหญ่ที่เกี่ยวกันแรงกว่าชุดข้อมูลอื่น) คู่ที่สัมพันธ์สูงสุดคือ

คู่ฟีเจอร์	r
MntMeatProducts – NumCatalogPurchases	0.724
MntWines – NumStorePurchases	0.642
MntWines – NumCatalogPurchases	0.635
MntFruits – MntFishProducts	0.594
Income – NumCatalogPurchases	0.589
Income – NumWebVisitsMonth	–0.553

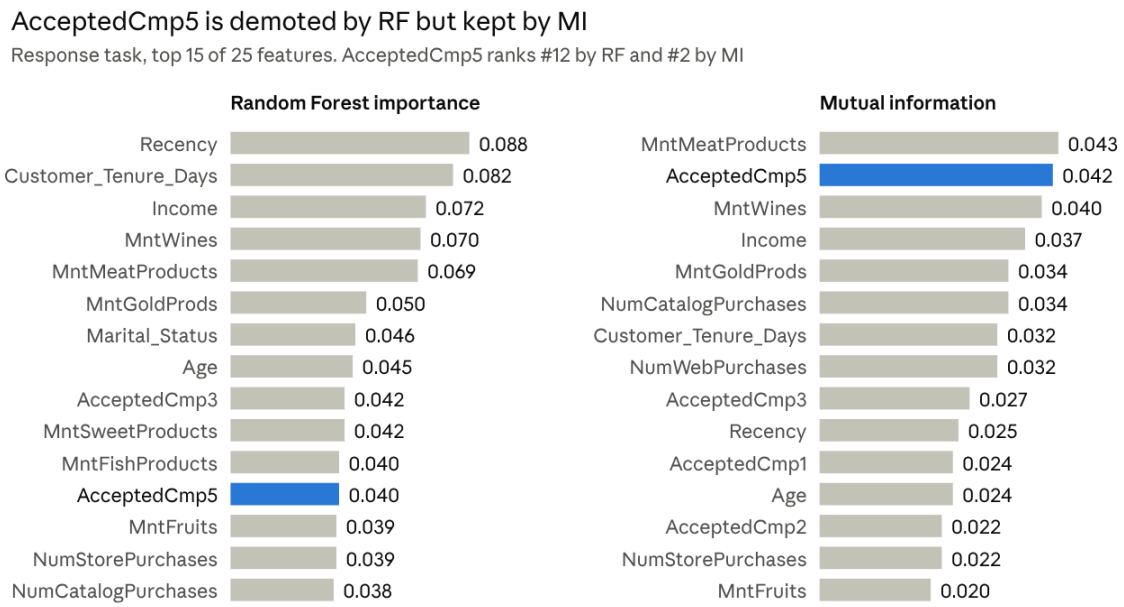
ตารางที่ **2.5** คู่ฟีเจอร์ที่มีค่าสหสัมพันธ์สูง

ฟีเจอร์ Income, MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds และ NumWebPurchases, NumCatalogPurchases, NumStorePurchases มีสหสัมพันธ์เชิงบวกกันทุกคู่ (Pearson r = 0.30–0.73) แสดงว่าลูกค้าที่มีรายได้สูงมักใช้จ่ายและซื้อสินค้ามากกว่า ฟีเจอร์กลุ่มนี้จึงมีข้อมูลซ้ำซ้อนกันบางส่วน ไม่ได้เป็นมิติที่เป็นอิสระต่อกัน Education กับ Marital_Status ไม่ซ้ำซ้อนกัน (Cramér's V = 0.043)

2.6 Feature Importance (RF) และ Mutual Information

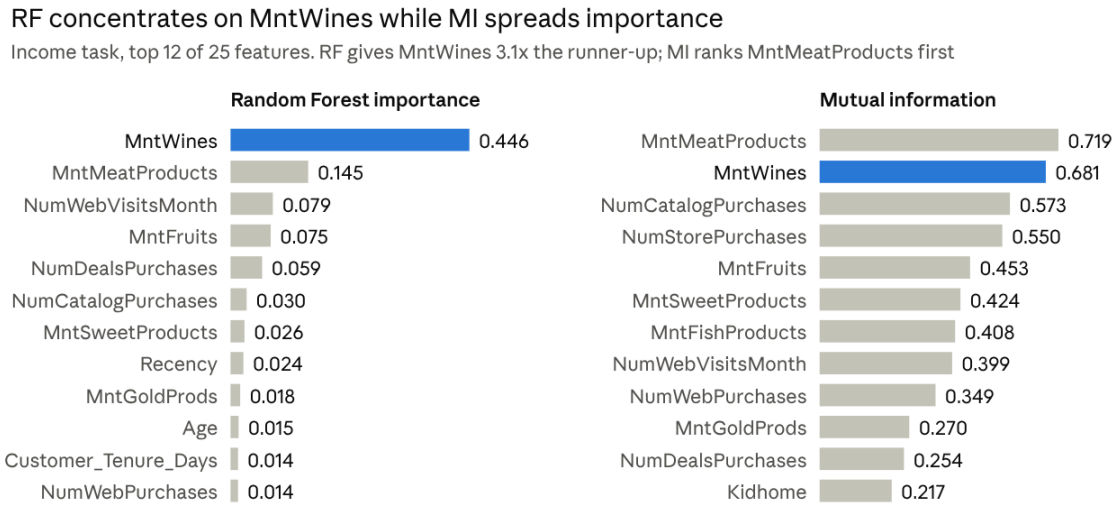
ตัววัด bivariate ดูทีละฟีเจอร์ จึงมองไม่เห็นปฏิสัมพันธ์และความสัมพันธ์ไม่เชิงเส้น เราจึงเพิ่มอีกสองมุมมองที่วัดสิ่งต่างกัน

- **Random Forest:** n_estimators = 300, random_state = 42 ความสำคัญรวมเท่ากับ 1 และ dummy ของ Education / Marital_Status ถูกรวมกลับเป็นคอลัมน์แม่
- **Mutual Information (MI):** mutual_info_classif / mutual_info_regression ฟีเจอร์ตัวเลขใช้ discrete_features=False ฟีเจอร์หมวดหมู่ใช้ True



ภาพที่ **2.1** ความสำคัญของฟีเจอร์จาก RF และ MI (classify)

- RF ดัน `Recency` (0.088) และ `Customer_Tenure_Days` (0.082) ขึ้นอันดับ 1–2 ทั้งที่ bivariate ให้เพียงอันดับ 7–8
- `AcceptedCmp5` ที่เป็นอันดับ 1 ของ bivariate ตกเป็นอันดับ 12 ใน RF (0.040) แต่ MI ยังให้อันดับ 2 (0.042)
- ทั้งสามวิธีจึงเห็นไม่ตรงกันว่า `AcceptedCmp1`, `AcceptedCmp2`, `AcceptedCmp3`, `AcceptedCmp4`, `AcceptedCmp5` ควรมีน้ำหนักเท่าใดเมื่อเทียบกับ `Recency` และ `MntWines`, `MntFruits`, `MntMeatProducts`, `MntFishProducts`, `MntSweetProducts`, `MntGoldProds` ไม่มีวิธีใดเป็นคำตอบสุดท้าย
- การที่ RF ลดอันดับ `AcceptedCmp1`, `AcceptedCmp2`, `AcceptedCmp3`, `AcceptedCmp4`, `AcceptedCmp5` เป็นผลของความซ้ำซ้อนระหว่างฟีเจอร์ในกลุ่มเดียวกัน ไม่ใช่หลักฐานว่าฟีเจอร์เหล่านี้ผิดปกติ



ภาพที่ **2.2** ความสำคัญของฟีเจอร์จาก RF และ MI (regression)

- RF ให้ MntWines 0.446 ซึ่งมากกว่าอันดับสอง MntMeatProducts (0.145) ถึง 3 เท่า
- NumCatalogPurchases ที่เป็นอันดับ 1 ของ bivariate ตกเป็นอันดับ 6 (0.030) เพราะมันสัมพันธ์กับ MntWines และ MntMeatProducts (r = 0.57–0.73) เมื่อ RF เลือกสองตัวนั้นแยกก่อน ข้อมูลที่เหลือให้ตัวอื่นก็น้อยลง
- MI เรียงลำดับคล้าย bivariate: MntMeatProducts (0.719) นำ MntWines (0.681) และ NumCatalogPurchases / NumStorePurchases ยังสูงทั้งที่ RF ลดอันดับ ยืนยันว่าเป็นผลของ RF ไม่ใช่ฟีเจอร์อ่อน
- ค่า MI สูงสุดของ regression (0.719) มากกว่า classify (0.043) ประมาณ 17 เท่า สอดคล้องกับข้อสังเกตว่า Income มีสัญญาณที่เรียนรู้ได้ชัดกว่า Response มาก
- Response ซึ่งผ่านเกณฑ์ bivariate (0.133) ได้ RF เพียง 0.0005 (อันดับ 22) และ MI 0.038 (อันดับ 21) แปลว่าเมื่อมีฟีเจอร์ MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds และ NumWebPurchases, NumCatalogPurchases, NumStorePurchases อยู่แล้ว Response แทบไม่เพิ่มข้อมูลใหม่

2.7 การโหวต 3 วิธีเพื่อเลือกฟีเจอร์ (three-vote rule)

คะแนนสามวิธีอยู่คละสเกล (bivariate เป็นค่ามีเครื่องหมาย –1 ถึง +1, RF รวมเท่ากับ 1, MI ไม่มีเพดาน) การเฉลี่ยจึงทำให้วิธีที่มีช่วงค่ากว้างที่สุดครองผล เราจึงให้แต่ละวิธีลงคะแนน “ผ่าน / ไม่ผ่าน” หนึ่งเสียงต่อฟีเจอร์

วิธี	ผ่านเมื่อ	classify	regression
Bivariate	ค่าสัมบูรณ์ ≥ 0.10	19 จาก 25	17 จาก 25
Random Forest	อยู่ใน top-N	N = 15	N = 12
Mutual Information	อยู่ใน top-N	N = 15	N = 12

ตารางที่ **2.6** เกณฑ์การคัดเลือกฟีเจอร์ของแต่ละวิธี

จำนวนโหวตแบ่งฟีเจอร์เป็น 4 ชั้น

ชั้น	ความหมาย	จำนวน classify	จำนวน regression
3/3	ทุกวิธีเห็นตรงกัน เลือก	10	8
2/3	สองในสามเห็นตรงกัน พิจารณารายตัว	8	5
1/3	วิธีเดียวผ่าน ตัด	3	7
0/3	ไม่ผ่านเลย ตัด	4	5

ตารางที่ **2.7** ระดับผลการโหวตและจำนวนฟีเจอร์

2.8 ผลการโหวตชั้น 3/3 และ 2/3

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
AcceptedCmp5	0.328 ✓	0.040 ✓ (12)	0.042 ✓ (2)
AcceptedCmp3	0.254 ✓	0.042 ✓ (9)	0.027 ✓ (9)
MntWines	0.247 ✓	0.070 ✓ (4)	0.040 ✓ (3)

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
MntMeatProducts	0.237 ✓	0.069 ✓ (5)	0.043 ✓ (1)
NumCatalogPurchases	0.221 ✓	0.038 ✓ (15)	0.034 ✓ (6)
Recency	0.199 ✓	0.088 ✓ (1)	0.025 ✓ (10)
Customer_Tenure_Days	0.194 ✓	0.082 ✓ (2)	0.032 ✓ (7)
MntGoldProds	0.141 ✓	0.050 ✓ (6)	0.034 ✓ (5)
Income	0.133 ✓	0.072 ✓ (3)	0.037 ✓ (4)
MntFruits	0.126 ✓	0.039 ✓ (13)	0.020 ✓ (15)

ตารางที่ **2.8** ฟีเจอร์ที่ผ่านเกณฑ์ 3 จาก 3 วิธี (classify)

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
AcceptedCmp1	0.294 ✓	0.033 × (17)	0.024 ✓ (11)
AcceptedCmp2	0.169 ✓	0.008 × (23)	0.022 ✓ (13)
Marital_Status	0.152 ✓	0.046 ✓ (7)	0.011 × (20)
NumWebPurchases	0.148 ✓	0.030 × (19)	0.032 ✓ (8)
MntSweetProducts	0.117 ✓	0.042 ✓ (10)	0.017 × (17)
MntFishProducts	0.111 ✓	0.040 ✓ (11)	0.006 × (21)
NumStorePurchases	0.039 ×	0.039 ✓ (14)	0.022 ✓ (14)
Age	0.018 ×	0.045 ✓ (8)	0.024 ✓ (12)

ตารางที่ **2.9** ฟีเจอร์ที่ผ่านเกณฑ์ 2 จาก 3 วิธี (classify)

ชั้น 1/3 (3 ตัว): AcceptedCmp4, Teenhome, Education · ชั้น 0/3 (4 ตัว): Kidhome, NumWebVisitsMonth, NumDealsPurchases, Complain

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
NumCatalogPurchases	0.589 ✓	0.030 ✓ (6)	0.573 ✓ (3)
MntMeatProducts	0.584 ✓	0.145 ✓ (2)	0.719 ✓ (1)
MntWines	0.578 ✓	0.446 ✓ (1)	0.681 ✓ (2)
NumWebVisitsMonth	−0.553 ✓	0.079 ✓ (3)	0.399 ✓ (8)
MntSweetProducts	0.441 ✓	0.026 ✓ (7)	0.424 ✓ (6)
MntFruits	0.430 ✓	0.075 ✓ (4)	0.453 ✓ (5)
NumWebPurchases	0.388 ✓	0.014 ✓ (12)	0.349 ✓ (9)
MntGoldProds	0.325 ✓	0.018 ✓ (9)	0.270 ✓ (10)

ตารางที่ **2.10** ฟีเจอร์ที่ผ่านเกณฑ์ 3 จาก 3 วิธี (regression)

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
NumStorePurchases	0.530 ✓	0.008 × (15)	0.550 ✓ (4)
MntFishProducts	0.439 ✓	0.012 × (13)	0.408 ✓ (7)
Kidhome	−0.428 ✓	0.004 × (19)	0.217 ✓ (12)
Age	0.163 ✓	0.015 ✓ (10)	0.130 × (14)
NumDealsPurchases	−0.083 ×	0.059 ✓ (5)	0.254 ✓ (11)

ตารางที่ **2.11** ฟีเจอร์ที่ผ่านเกณฑ์ 2 จาก 3 วิธี (regression)

ชั้น 1/3 (7 ตัว): AcceptedCmp5, AcceptedCmp1, Education, AcceptedCmp4, Response, Customer_Tenure_Days, Recency · ชั้น 0/3 (5 ตัว): AcceptedCmp2, Marital_Status, Complain, Teenhome, AcceptedCmp3

สิ่งที่ตารางแสดง:

- ชั้น 2/3 ที่ตกเพียง bivariate (NumStorePurchases, Age ใน classify · NumDealsPurchases ใน regression) ผ่านทั้ง RF และ MI จึงมีสัญญาณไม่เชิงเส้นที่ค่าสหสัมพันธ์เชิงเส้นมองไม่เห็น

- ชั้น 2/3 ที่ตกเพียง RF (NumStorePurchases, MntFishProducts, Kidhome ใน regression) เป็นผลของ greedy splitting ในกลุ่มฟีเจอร์ที่สัมพันธ์กัน (หัวข้อ 2.6)
- ชั้น 2/3 ที่ตกเพียง MI (MntSweetProducts, MntFishProducts ใน classify) อาจเกิดจากตัวประมาณ MI ของเป้าหมายทวินารีซึ่งมีความแปรปรวนสูงกว่า ผลพลาดชิดเกณฑ์จึงไม่เป็นหลักฐานที่แข็งแกร่ง
- AcceptedCmp1 / AcceptedCmp2 ตกที่ RF ทั้งคู่ สอดคล้องกับการที่ RF ลดอันดับทั้งกลุ่ม AcceptedCmp1, AcceptedCmp2, AcceptedCmp3, AcceptedCmp4, AcceptedCmp5

2.9 Feature Interaction (ฟีเจอร์ที่ส่งผลร่วมกัน)

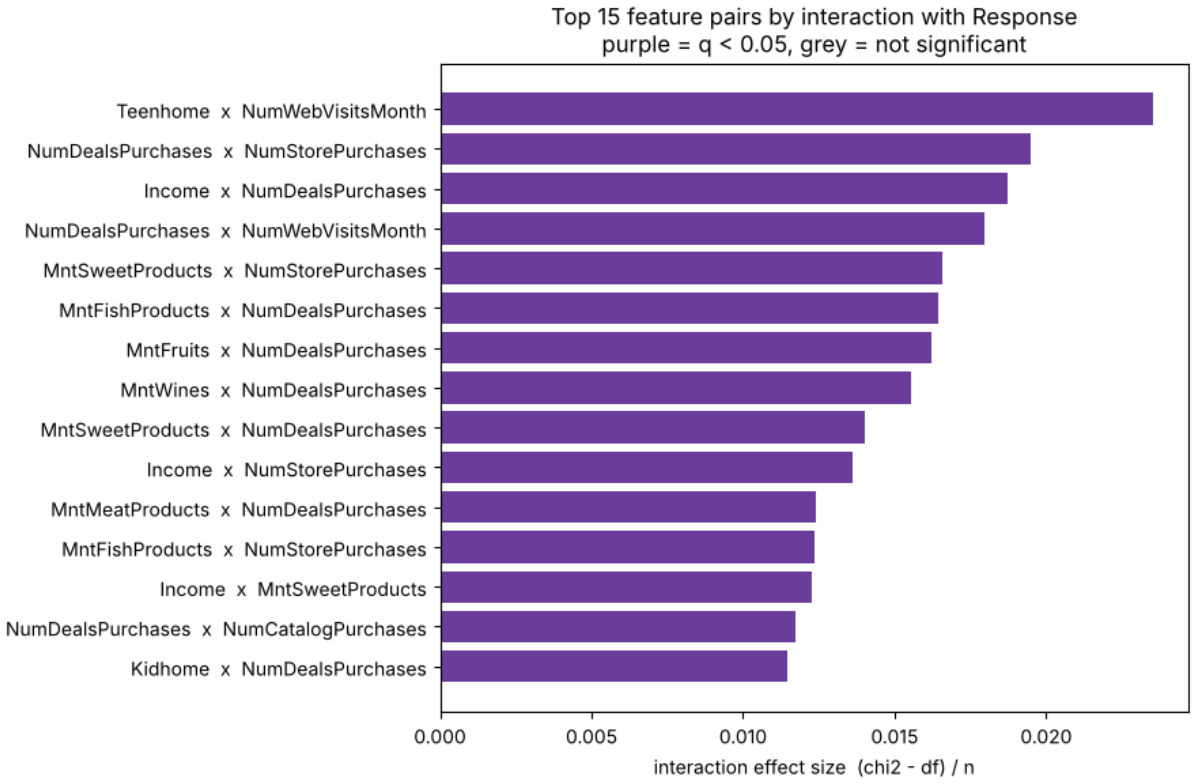
หัวข้อ 2.4–2.8 วัดฟีเจอร์ทีละตัว หัวข้อนี้ถามว่า ผลของฟีเจอร์ **A** ต่อเป้าหมายเปลี่ยนไปตามค่าของฟีเจอร์ **B** หรือไม่ โดยทดสอบทุกคู่จาก 25 ตัวแปร (296 คู่) บน 2,213 แถว

- ฟีเจอร์ตัวเลขแบ่งเป็น 4 ช่วงตามควอร์ไทล์ ฟีเจอร์ที่มีค่าน้อยใช้ค่าเดิม
- classify (Response): G^2 test [25] เทียบ log-linear model ที่มีเฉพาะคู่ [AB] [AY][BY] กับ model เต็ม ถ้าต่างกันมาก แปลว่าอัตราการตอบรับขึ้นกับ A และ B ร่วมกัน
- regression (Income): two-way ANOVA บน log1p(Income) ทดสอบเทอม A×B
- ขนาดผล = (สถิติทดสอบ – df) / n เพื่อเทียบคู่ที่มี df ต่างกันได้
- แก้การทดสอบหลายครั้งด้วย Benjamini–Hochberg [22] เก็บคู่ที่ $q < 0.05$

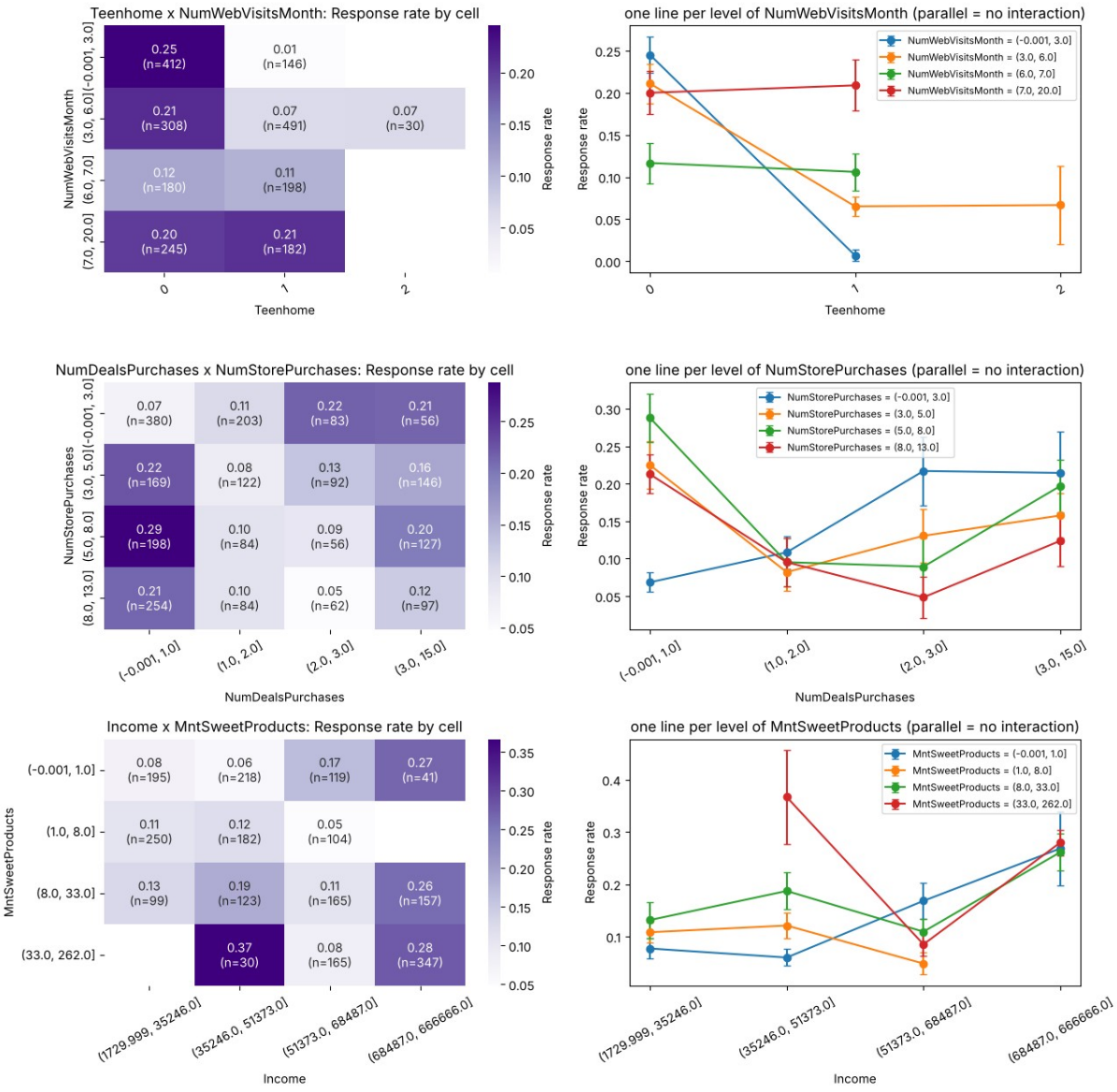
$$G^2 = 2 \sum_{a,b,y} n_{aby} \ln \frac{n_{aby}}{\hat{m}_{aby}}, \qquad \text{effect} = \frac{G^2 - df}{n}$$

	classify (Response)	regression (Income)
คู่ที่มีนัยสำคัญ (q < 0.05)	**33** จาก **296**	**156** จาก **296**
คู่ที่แรงที่สุด	Teenhome × NumWebVisitsMonth (0.024)	Teenhome × NumCatalogPurchases (0.115)
ใน 15 คู่บนสุด มีฟีเจอร์ที่ถูกตัดในหัวข้อ 2.7	11 คู่	10 คู่
ฟีเจอร์ที่ปรากฏบ่อยที่สุด	NumDealsPurchases (12 คู่)	MntWines (20 คู่), Teenhome (17 คู่)

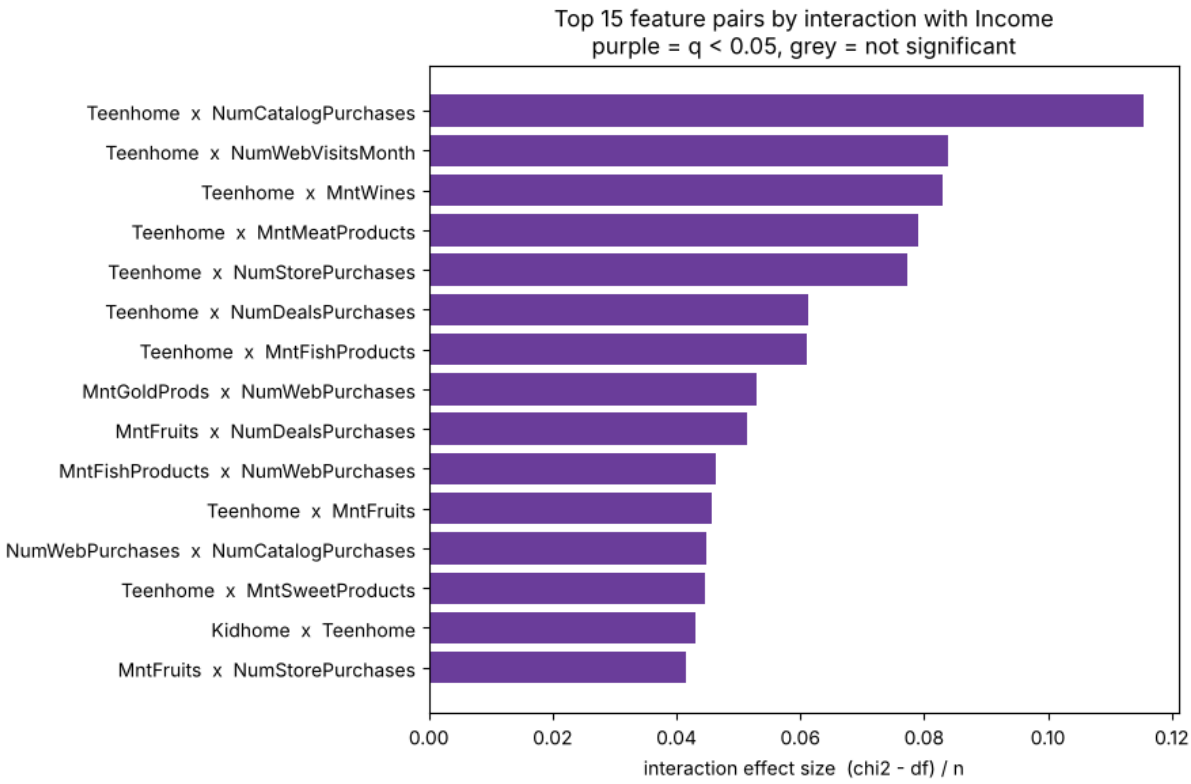
ตารางที่ **2.12** สรุปผลการทดสอบ Feature Interaction



ภาพที่ **2.3** คู่ฟีเจอร์ที่มี interaction สูงสุด 15 อันดับ (classify)



ภาพที่ **2.4** ตัวอย่าง interaction ระหว่างคู่ฟีเจอร์ (classify)



ภาพที่ **2.5** คู่ฟีเจอร์ที่มี interaction สูงสุด 15 อันดับ (regression)

- Teenhome × NumWebVisitsMonth: บ้านที่ไม่มีวัยรุ่นและเข้าเว็บน้อย ตอบรับ 25% แต่บ้านที่มีวัยรุ่นและเข้าเว็บน้อยตอบรับเพียง 1%
- ฟีเจอร์ที่ถูกตัดเพราะผลรายตัวอ่อน (Teenhome, NumDealsPurchases, NumWebVisitsMonth) กลับเป็นตัว "ปรับ" ผลของฟีเจอร์อื่น
- regression: 7 คู่บนสุดล้วนมี Teenhome แปลว่าความสัมพันธ์ระหว่างการซื้อกับรายได้เปลี่ยนไปตามจำนวนวัยรุ่นในบ้าน
- การทดสอบนี้ใช้ข้อมูลทุกแถวก่อนแบ่ง train/test เป็นการสำรวจข้อมูล ไม่ใช่คะแนนของโมเดล

บทที่ **3** ระเบียบวิธีและการตั้งค่าการทดลอง

ทุกโมเดลในบทที่ 4 ใช้ชุดข้อมูล การแบ่ง และตัววัดชุดเดียวกัน เพื่อให้สามารถเปรียบเทียบระหว่างเวอร์ชัน บทนี้สรุปกติกาที่ใช้ร่วมกัน รวมถึงข้อจำกัดของวิธีประเมิน

3.1 สิ่งที่ใช้ร่วมกัน

รายการ	classify (Response)	regression (Income)
จำนวนแถว	2,236	2,212
การแบ่งชุดข้อมูล	80/20, random_state = 42, stratified	80/20, random_state = 42
จำนวนแถวชุดทดสอบ	448 (บวก 67)	443
Input Shape	22	13
Main Metric	**F1**	**R^2**
Sub Metrics	ROC-AUC, precision, recall, accuracy, BCE	RMSE, MAE, MSE

ตารางที่ **3.1** การตั้งค่าที่ใช้ร่วมกันทุกโมเดล

3.2 การเตรียม **Data**

- StandardScaler fit เฉพาะชุด train แล้วใช้กับชุด test เพื่อไม่ให้ข้อมูลชุด test รั่วเข้าการเทรน
- classify: Income ที่หาย 24 แถวถูกเติมด้วยมัธยฐานของชุด train (51,651) ในฐานะฟีเจอร์
- Marital_Status ทำ one-hot 5 มิติ ทำให้ classify มี 22 มิติ regression ไม่ใช้ฟีเจอร์หมวดหมู่จึงคง 13 มิติ
- ไม่ทำ data augmentation
- classify ใช้ weighted BCE ด้วย pos_weight = neg / pos ≈ 5.7 เพื่อชดเชยความไม่สมดุล (ค่า BCE ที่รายงานคือ unweighted mean BCE)

3.3 แนวคิด **Library + Scratch**

ทุกโมเดลจะมี 2 version เสมอ

- **Library:** สร้างด้วย libraryใช้เป็นคำตอบอ้างอิง
- **Scratch:** เขียนเองด้วยทั้ง forward, backward และ optimizer จากสมการชุดเดียวกัน

ผลที่ได้จากโค้ดที่เขียนเองเชื่อถือได้เมื่อตรงกับ Library และความต่างที่เหลือมาจากการสุ่ม (ค่าเริ่มต้น ลำดับข้อมูล) ไม่ใช่ข้อผิดพลาดของสมการ

3.4 การตั้งค่าการฝึกแต่ละกลุ่มเวอร์ชัน

เวอร์ชัน	Optimizer	จำนวนรอบ	การควบคุมอื่น
V1.0	Adam (lr 1e-3, β = 0.9 / 0.999)	50 epochs, batch 64	ไม่มี schedule ไม่มี weight decay เก็บรอบสุดท้าย
V1.1, V1.2	Adam (lr 1e-3) + cosine annealing	100 epochs	weight decay 1e-4, BatchNorm, dropout, residual · เก็บรอบสุดท้าย
V2.0	Adam (lr 1e-3) + cosine (T_{max} = 100)	100 epochs	weight decay 1e-4 · เก็บ checkpoint ที่ F1 ดีสุด (classify) หรือ RMSE ต่ำสุด (regression)
V3.0	Gradient boosting (ต้นไม้ CART)	จำนวนต้นไม้ให้ early stopping	ความลึกสูงสุด 5, min leaf 20 · checkpoint ที่ BCE ต่ำสุด
V1.3, V2.1, V3.1	เหมือน V1.1, V2.0, V3.0 ตามลำดับ	เหมือนเดิม	ใช้ค่าเดิมทั้งหมด เปลี่ยนแค่วิธีแบ่งข้อมูลเป็น 5-fold
V4, V5 (neural network)	Adam (lr 1e-3) + cosine (T_{max} = 100), batch 64	สูงสุด 100 epochs เลือกจำนวน epoch จาก inner 3-fold	weight decay 1e-4 · ฝึกใหม่บนชุด train ทั้งหมด · threshold จาก inner out-of-fold · 15 outer folds

เวอร์ชัน	Optimizer	จำนวนรอบ	การควบคุมอื่น
V5–V7 (โมเดลจาก Library)	Optuna (TPE) 30 trials (Stacking 10) บน inner 3-fold	Optuna เลือกเป็น hyper-parameter (เช่น จำนวนต้นไม้)	threshold ที่ F1 สูงสุดจาก inner out-of-fold · V7 จำกัดความลึกของ LightGBM เป็น 1, 2 และไม่จำกัด · 15 outer folds

ตารางที่ **3.2** การตั้งค่าการฝึกของแต่ละกลุ่มเวอร์ชัน

3.5 ตัววัดผล

classify ใช้ F1 เป็นตัวหลัก เพราะคลาสบวกมีเพียง 14.9% F1 คือค่าเฉลี่ยฮาร์โมนิกของ precision และ recall ซึ่งคำนวณจาก TP, FP, FN เท่านั้น

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}$$

regression ใช้ R^2 เป็นตัวหลัก คือสัดส่วนความแปรปรวนของรายได้ที่โมเดลอธิบายได้ ส่วน RMSE แสดงเป็นหน่วยดอลลาร์เพื่อให้เห็นขนาดความคลาดเคลื่อน

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$$

3.6 การประเมินแบบ **k-fold** และข้อจำกัดของวิธีประเมิน

ชุด test มีเพียง 448 แถวใน classify (ลูกค้าที่ตอบรับ 67 แถว) และ 443 แถวใน regression ผลจาก split เดียวจึงสั่นไหวได้ง่าย จึงเพิ่มการตรวจ k-fold ในสามเวอร์ชันหลัง

- k = 5, shuffle = True, random_state = 42 (classify - StratifiedKFold, regression - KFold) ทำบนชุดที่ clean แล้วทั้งชุด
- V1.3 ฝึกคงที่ 100 epochs ต่อ fold ไม่เลือก checkpoint เหมือน V1.1 บน split เดียว
- V2.1 และ V3.1 เลือก checkpoint ต่อ fold เหมือน V2.0 และ V3.0 ผลรายงานเป็นค่าเฉลี่ย ± ส่วนเบี่ยงเบนมาตรฐานของสองฝั่ง (Library และ scratch)

3.7 timeline การพัฒนา

ลำดับสร้าง	เวอร์ชัน	สิ่งที่เพิ่ม
1	V1.0	Baseline: MLP 1 hidden layer
2	V1.1	MLP หลาย Layer แบบ ResNet (BatchNorm, dropout, residual, cosine schedule)
3	V1.2	ตัดชั้น 2/3 เหลือเฉพาะ 3/3
4	V2.0	architecture ใหม่: FT-Transformer-lite (classify) และ NAM (regression)
5	V3.0	Gradient boosting (LightGBM + ประมวลผลเอง) เป็นตัวเทียบที่ดีที่สุด
6	V1.3	5-fold: MLP หลาย Layer
7	V2.1	5-fold: architecture ใหม่
8	V3.1	5-fold: gradient boosting
9	V4.0, V4.1	ReGLU ใน FT-Transformer-lite และ NAM (split เดียว + 15 folds)
10	V5.0, V5.1	ฟีเจอร์ interaction และ NA²M
11	V6.0, V6.1	โมเดลในหลักสูตรทั้งหมด (classify 19 ตัว, regression 14 ตัว) + Optuna (15 folds แบบ nested)
12	V7.0, V7.1	หาสาเหตุของ gap (บันได interaction, learning curve) และภาพอธิบาย NAM

ตารางที่ **3.3** ลำดับการพัฒนาโมเดล

3.8 การทดลองรอบที่สอง (V4–V7): คำถาม 3 ข้อจากอาจารย์

หลังนำเสนอ อาจารย์ถาม 3 ข้อ รอบที่สองตอบทั้งสามข้อด้วยกติกาเดียวกันทุกโมเดล เพื่อให้เทียบกันได้ตรงๆ

คำถาม	เวอร์ชัน	สิ่งที่ทำ
1. Feature interaction	หัวข้อ 2.9, V5	หาคู่ที่ส่งผลร่วมกัน แล้วใส่คู่เหล่านั้นเข้าโมเดล
2. ทำไม FT-Transformer ใช้ ReGLU และช่วยเราไหม	V4	เปลี่ยน FFN จาก ReLU เป็น ReGLU ทั้ง FT-Transformer-lite และ NAM
3. extracurricular ชนะจริงหรือไม่	V6, V7	รันโมเดลในหลักสูตรทุกตัวบน fold เดียวกัน แล้วหาสาเหตุของ gap

ตารางที่ **3.4** คำถามจากอาจารย์และเวอร์ชันที่ตอบ

กติการ่วมของทุกโมเดลในรอบที่สอง

- ข้อมูล ฟีเจอร์ และการเตรียม Data เหมือนหัวข้อ 3.1–3.2
- **Repeated 5-fold × 3 seeds = 15 outer folds** ชุดเดียวกันทุกโมเดล (seed 42, 43, 44)
- โมเดลจาก library ปรับ hyper-parameter ด้วย Optuna [19] (TPE) 30 trials (Stacking 10 trials) บน inner 3-fold ของชุด train เท่านั้น
- เป้าหมายของ Optuna: average precision (classify) และ RMSE (regression)
- classify เลือก threshold ที่ให้ F1 สูงสุดจาก inner out-of-fold ไม่ใช้ 0.5 คงที่
- neural network เลือกจำนวน epoch จาก inner 3-fold แล้วฝึกใหม่บนชุด train ทั้งหมด ไม่เลือก checkpoint บน fold ที่ใช้รายงานอีก (แก้ปัญหาของ V2.0/V2.1)
- FT-Transformer-lite และ NAM ใช้ hyper-parameter เดิมจาก V2.0 ไม่ได้ผ่าน Optuna
- ทุกโมเดล neural network มีเวอร์ชัน Scratch ที่ตรวจ forward, loss และ gradient กับ PyTorch บนน้ำหนักชุดเดียวกัน

- เปลี่ยนจากแผนเดิม: inner fold ลดจาก 5 เป็น 3 เพราะเวลาคำนวณ

ขั้นตอนการปรับ **hyper-parameter** ใน **1 outer fold**

1. แบ่งข้อมูลเป็นชุด train (80%) และชุด test (20%)
2. โมเดลจาก Library: Optuna ลอง hyper-parameter 30 ชุด แต่ละชุดวัดผลด้วย inner 3-fold บนชุด train เท่านั้น
3. นำชุดที่ดีที่สุดมาฝึกใหม่บนชุด train ทั้งหมด
4. classify: เลือก threshold ที่ให้ F1 สูงสุดจาก inner out-of-fold
5. neural network (PyTorch): ไม่ใช้ Optuna ใช้ hyper-parameter คงที่จาก V2.0 และเลือกเฉพาะจำนวน epoch และ threshold จาก inner 3-fold
6. Scratch: คัดลอกจำนวน epoch และ threshold ที่เวอร์ชัน PyTorch เลือกไว้มาใช้โดยไม่ค้นหาใหม่ แล้วฝึกน้ำหนักเองตั้งแต่ต้น
7. วัดผลบนชุด test ของ fold นั้นเพียงครั้งเดียว

การทดสอบแบบจับคู่ **(paired)**

ทุกโมเดลถูกวัดบน fold เดียวกัน จึงเทียบผลต่างทีละ fold (d_j) ได้ ใช้ 3 วิธีร่วมกัน

- Wilcoxon signed-rank [24] บน d_j 15 ค่า (ไม่สนใจว่า fold ซ้อนกัน จึงมองโลกในแง่ดี)
- Nadeau–Bengio corrected t-test [20] ชดเชยที่ชุด train ของแต่ละ fold ซ้อนกัน (เข้มกว่า)
- Bootstrap 95% CI ของค่าเฉลี่ย d_j (5,000 ครั้ง)

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s_d^2}}, \qquad J = 15, \quad \frac{n_{\text{test}}}{n_{\text{train}}} = \frac{1}{4}$$

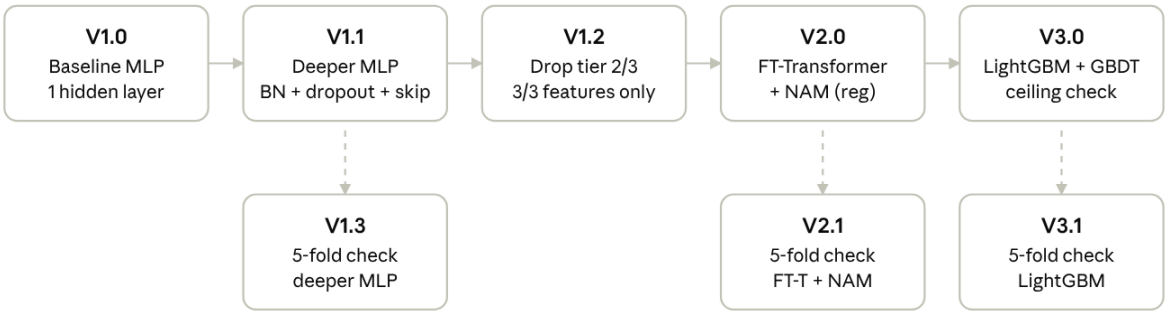
ผลทุกตัวเก็บใน `results/ledger.csv` (หนึ่งแถวต่อโมเดลต่อ fold) และตัวเลขในบทที่ 4–5 คำนวณจากไฟล์นี้

บทที่ **4** วิวัฒนาการของโมเดลตามลำดับเวอร์ชัน

บทนี้เล่าเหตุผลของทุกการตัดสินใจตามลำดับเวอร์ชัน แต่ละหัวข้อมี 5 ขั้นเสมอ

1. คำถาม ที่เวอร์ชันนี้ต้องตอบ
2. **architecture** และการตั้งค่าที่เปลี่ยนไป

3. ผลลัพธ์ บนชุดทดสอบชุดเดียวกัน (n = 448 classify · 443 regression) ทั้ง PyTorch (Library) และ Scratch (NumPy)
4. ทำไมจึงเป็นเช่นนั้น อธิบายผลด้วยหลักฐาน
5. คำถามถัดไป ที่ผลนั้นทิ้งไว้

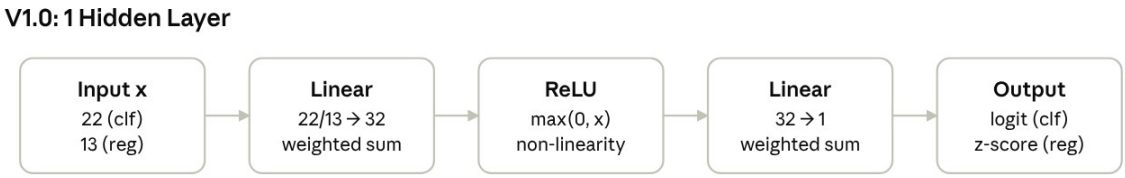


ภาพที่ **4.1** ลำดับเวอร์ชันของโมเดล

4.1 V1.0 — Baseline: MLP 1 Layer

1) คำถาม**:** โมเดลที่ง่ายที่สุดทำได้ดีแค่ไหน

2) architecture



ภาพที่ **4.2** architecture ของ V1.0

- Initial Weight แบบ He
- Adam (lr 1e-3)
- 50 epochs
- batch 64
- seed 42

- regression ฝึกบน Income ที่ทำ z-score และคำนวณทุกตัวชี้วัดหลังแปลงกลับเป็นดอลลาร์

3) ผลลัพธ์

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.5161**	**0.4974**	**0.0187**
ROC-AUC	0.8669	0.8587	0.0082
Precision	0.4034	0.3852	0.0182
Recall	0.7164	0.7015	0.0149
Accuracy	0.7991	0.7879	0.0112
BCE	0.4028	0.4426	0.0398

ตารางที่ **4.1** ผลลัพธ์ V1.0 (classify)

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7423**	**0.7695**	**0.0272**
RMSE ($)	11,064.7	10,463.3	5.4%
MAE ($)	7,067.9	6,543.3	7.4%

ตารางที่ **4.2** ผลลัพธ์ V1.0 (regression)

4) ทำไมจึงเป็นเช่นนั้น

- Accuracy 0.799 ต่ำกว่า การเดา "ไม่ตอบรับ" ทุกแถว (0.851) เพราะการถ่วงน้ำหนักคลาสบวก (pos_weight ≈ 5.7) เพิ่ม recall ขึ้นเป็น 0.72 แลกกับ precision 0.40
- อันดับของลูกค้าถูกขับด้วย AcceptedCmp1, AcceptedCmp2, AcceptedCmp3, AcceptedCmp4, AcceptedCmp5, MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds, Recency และ Customer_Tenure_Days ตรงกับรายชื่อฟีเจอร์ใน EDA

- regression ได้ R^2 0.74–0.77 สูงตามที่ EDA คาด คอลัมน์ MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds เป็นตัวแทนของ Income ในเชิงกลไก
- Scratch ได้ R^2 สูงกว่า Library 0.027 ซึ่งเป็นความแปรปรวนจากค่าเริ่มต้น ไม่ใช่ความได้เปรียบของวิธีเขียนเอง

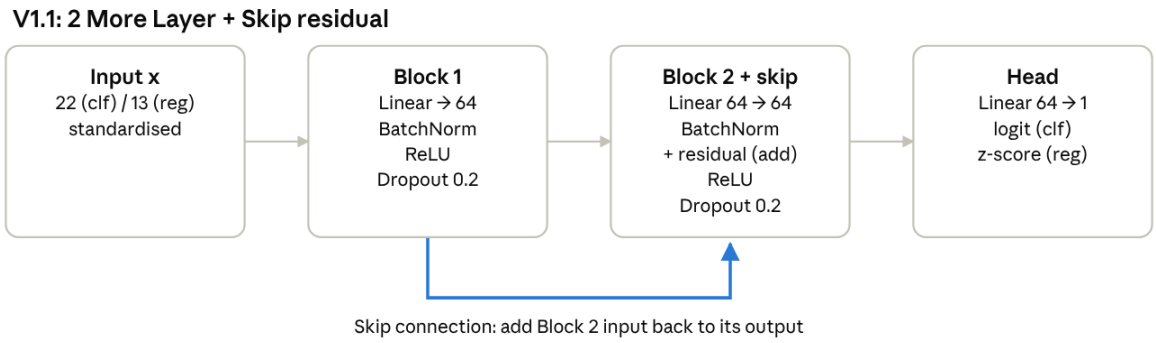
5) คำถามถัดไป

- Loss ลดลงเรื่อย ๆ ไม่มีจุดหัก แสดงว่าโมเดลยัง under-fit
- V1.0 เป็นแค่ baseline ไว้เทียบ ต้องลองทำ MLP ให้มีหลาย Layer แล้วดูว่าคะแนนดีกว่า baseline ไหม → V1.1

4.2 V1.1 — MLP แบบ ResNet

1) คำถาม**:** MLP ที่มีหลาย Layer ให้คะแนนดีกว่า baseline V1.0 (1 hidden layer) หรือไม่ โดยเปลี่ยนแค่จำนวน Layer regularization และ schedule

2) architecture



ภาพที่ **4.3** architecture ของ V1.1

- BatchNorm: ปรับสเกลการเปิดใช้งานของแต่ละหน่วยข้าม batch ทำให้ฝึกนิ่งขึ้น
- Dropout 0.2: สุ่มปิดหน่วยระหว่างฝึก ลด overfitting
- Residual: บวกอินพุตของบล็อกกลับเข้าไป ทำให้เพิ่ม Layer ได้อย่างปลอดภัย
- Adam (lr 1e-3)
- Cosine annealing

- Weight decay 1e-4
- 100 epochs

3) ผลลัพธ์

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.5455**	**0.5158**	**0.0297**
ROC-AUC	0.8818	0.8625	0.0193
Precision	0.4122	0.3984	0.0138
Recall	0.8060	0.7313	0.0747
Accuracy	0.7991	0.7946	0.0045
BCE	0.3966	0.4121	0.0155

ตารางที่ **4.3** ผลลัพธ์ V1.1 (classify)

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7774**	**0.7595**	**0.0179**
RMSE ($)	10,283.8	10,688.1	3.9%
MAE ($)	6,523.7	6,642.7	1.8%

ตารางที่ **4.4** ผลลัพธ์ V1.1 (regression)

ตัววัด	V1.0	V1.1	เปลี่ยน
F1 · PyTorch	0.5161	0.5455	+0.0294
F1 · Scratch	0.4974	0.5158	+0.0184
R^2 · PyTorch	0.7423	0.7774	+0.0351
R^2 · Scratch	0.7695	0.7595	–0.0100

ตารางที่ **4.5** เปรียบเทียบผล V1.0 กับ V1.1

4) ทำไมจึงเป็นเช่นนั้น

- classify: การเพิ่ม Layer ทำให้ recall ของ Library เพิ่มจาก 0.72 เป็น 0.81 โดย precision แทบไม่เสีย (0.40 → 0.41) F1 จึงเพิ่ม
- regression: Library ดีขึ้นชัด (R^2 +0.035, RMSE −7%) แต่ Scratch ลดลง 0.010 ซึ่งอยู่ในช่วงความผันผวนจากค่าเริ่มต้น
- สรุปได้ว่าการเพิ่ม Layer ช่วยให้ดีกว่า baseline V1.0 บน classify และบน regression

5) คำถามถัดไป

- ชั้น 2/3 ที่เก็บไว้มีสัญญาณจริงหรือเป็นแค่น้ำหนักที่ไม่จำเป็น → ทดลองตัดทิ้งใน V1.2

4.3 V1.2 — ตัดชั้น 2/3 เหลือเฉพาะ 3/3

1) คำถาม**:** ฟีเจอร์ชั้น 2/3 (classify 8 ตัว · regression 5 ตัว) มีสัญญาณจริง หรือเป็นแค่น้ำหนักที่ตัดออกได้ (ตามหัวข้อ 2.7 ชั้นนี้ให้พิจารณารายตัว จึงใช้การทดลองตัดสินแทน)

2) architecture: เหมือน V1.1 ทุกประการ (โครงในรูปที่ 4.2, ชุดแบ่ง และ seed เดิม) เปลี่ยนแค่จำนวนอินพุต

- classify: อินพุต 22 → 10 มิติ (ตัด one-hot ของ `Marital_Status` ทิ้ง)
- classify: พารามิเตอร์ 5,185
- classify: ฟีเจอร์ที่ถูกตัด `AcceptedCmp1`, `AcceptedCmp2`, `Marital_Status`, `NumWebPurchases`, `MntSweetProducts`, `MntFishProducts`, `NumStorePurchases`, `Age`
- regression: อินพุต 13 → 8 มิติ
- regression: พารามิเตอร์ 5,057
- regression: ฟีเจอร์ที่ถูกตัด `NumStorePurchases`, `MntFishProducts`, `Kidhome`, `Age`, `NumDealsPurchases`

3) ผลลัพธ์

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.4566**	**0.4645**	**0.0079**
ROC-AUC	0.8452	0.8463	0.0011
Precision	0.3289	0.3403	0.0114
Recall	0.7463	0.7313	0.0150
Accuracy	0.7344	0.7478	0.0134
BCE	0.4712	0.4471	0.0241

ตารางที่ **4.6** ผลลัพธ์ V1.2 (classify)

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7562**	**0.7586**	**0.0024**
RMSE ($)	10,761.2	10,708.1	0.5%
MAE ($)	6,982.8	6,935.8	0.7%

ตารางที่ **4.7** ผลลัพธ์ V1.2 (regression)

ตัววัด	V1.1	V1.2	เปลี่ยน
F1 · PyTorch	0.5455	0.4566	–0.0889
F1 · Scratch	0.5158	0.4645	–0.0513
R^2 · PyTorch	0.7774	0.7562	–0.0212
R^2 · Scratch	0.7595	0.7586	–0.0009

ตารางที่ **4.8** เปรียบเทียบผล V1.1 กับ V1.2

4) ทำไมจึงเป็นเช่นนั้น

- classify: F1 ลดลงทิศทางเดียวกันทั้งคู่ ชี้ว่าชั้น 2/3 ส่งผลจริงไม่ซ้ำซ้อน
- feature ที่น่าจะสำคัญที่สุดคือ `AcceptedCmp1` (bivariate 0.294 สูงสุดในที่ถูกตัด) ร่วมกับ non linear feature ของ `Marital_Status`, `NumStorePurchases`, `Age` นี่เป็นข้อสรุปที่ไม่ได้แยกทดสอบทีละตัว

- regression: เมื่อใช้เฉพาะชุด 3/3 (8 ตัว) Library เสีย R^2 เพียง 0.021 และ Scratch แทบไม่เปลี่ยน (–0.001) แสดงว่าฟีเจอร์อีก 5 ตัวที่ถูกตัดเพิ่มข้อมูลให้โมเดลน้อยมาก ชุด 3/3 คือ `MntWines`, `MntMeatProducts`, `MntFruits`, `MntSweetProducts`, `MntGoldProds`, `NumCatalogPurchases`, `NumWebPurchases` และ `NumWebVisitsMonth`
- ความไม่เท่ากันระหว่างสองงานนี้ชี้ว่าความซ้ำซ้อนของฟีเจอร์ regression สูงกว่า classify มาก (ตาม EDA)

5) คำถามถัดไป

- ชั้น 2/3 กลับมาใช้ในทุกโมเดลถัดไป (18 ฟีเจอร์ใน classify และ 13 ฟีเจอร์ใน regression)
- regression ชุด 8 มิติได้ผลใกล้เคียงกันและเล็กกว่า จึงเก็บไว้เป็นทางเลือก แต่ยังใช้ชุด 13 มิติเป็นหลัก
- คำถามใหม่: outside curriculum architecture ให้คะแนนบนชุดฟีเจอร์เดียวกันได้ดีกว่าหรือไม่ → V2.0

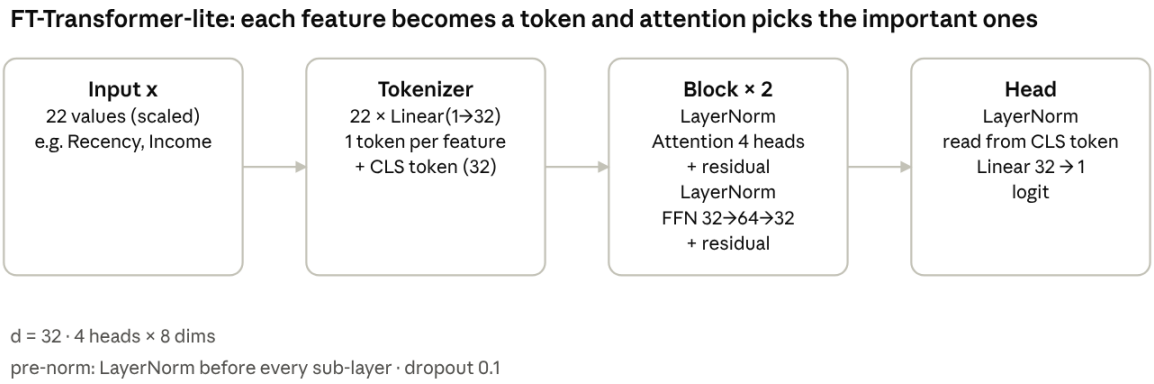
4.4 V2.0 — outside curriculum architecture: FT-Transformer-lite และ NAM

1) คำถาม**:** architecture ที่ไม่อยู่ในรายชื่อที่เรียน (ไม่ใช่ Linear/Logistic, Tree, RF, GB, NB, SVM, kNN, k-means, Perceptron, MLP) จะให้คะแนนดีกว่า MLP หลาย Layer หรือไม่ และจะช่วยอธิบายผลได้หรือไม่

- classify: **FT-Transformer-lite**
- regression: **NAM** (Neural Additive Model) พร้อม `log1p` + Huber (ทุกฟีเจอร์มี neural network เล็ก ๆ ของตัวเองแล้วรวมผล)

2) architecture และการทำงาน

4.4.1 FT-Transformer-lite (classify)



ภาพที่ **4.4** architecture ของ FT-Transformer-lite

การทำงานทีละขั้น:

1. **Tokenizer:** แต่ละฟีเจอร์ x_j ผ่าน Linear(1→32) ของตัวเอง ได้เวกเตอร์ 32 มิติ
2. ได้ 22 token ตามจำนวนฟีเจอร์ บวก CLS token ที่เรียนรู้ได้ 1 ตัว รวม 23 token
3. **Self-attention (4 หัว):** ทุก token สร้าง Q, K, V ด้วย Linear 32→32
4. คะแนนความสัมพันธ์กับ token อื่นทั้งหมดผ่าน softmax (รวมเป็น 1)
5. เฉลี่ย V ตามน้ำหนักนั้น
6. **FFN:** แต่ละ token ผ่าน Linear 32→64, ReLU, Linear 64→32
7. **Residual:** ทุก sub-layer บวกอินพุตกลับเข้าไป
8. ซ้ำ 2 บล็อก
9. **Head:** ใช้เฉพาะ CLS token สุดท้าย
10. ผ่าน LayerNorm
11. ผ่าน Linear 32→1 ได้ logit

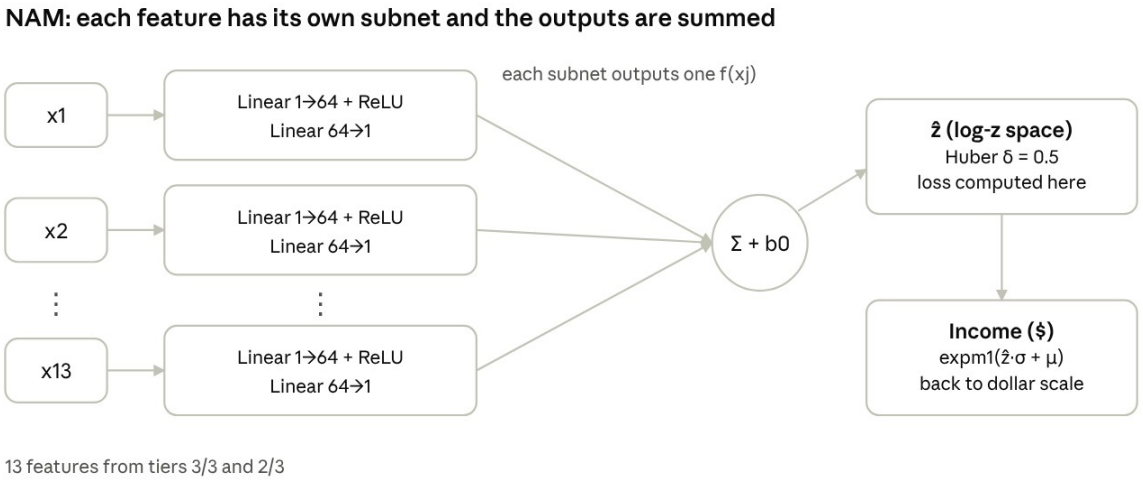
$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V, \qquad d_h = \frac{32}{4} = 8$$

- แถวของ CLS ในเมทริกซ์น้ำหนักคือ “CLS อ่านสัญญาณจากแต่ละฟีเจอร์เท่าไร” จึงใช้เป็นค่าความสำคัญของฟีเจอร์ได้
- พารามิเตอร์รวม: tokenizer 1,408 + CLS 32 + 2 บล็อก × 8,544 + LayerNorm สุดท้าย 64 + head 33 = 18,625

- Weighted BCE (pos_weight ≈ 5.7)
- Adam (lr 1e-3) + cosine
- Weight decay 1e-4
- 100 epochs
- เก็บ checkpoint ที่ F1 ดีที่สุด

คำว่า **"lite":** FT-Transformer ต้นฉบับ [1] ใช้ token 192 มิติ, 3 บล็อก, 8 หัว attention และ FFN แบบ ReGLU ส่วนเวอร์ชันของเราย่อเหลือ 32 มิติ, 2 บล็อก, 4 หัว และ FFN แบบ ReLU (18,625 พารามิเตอร์) ให้เหมาะกับข้อมูลราว 2,200 แถว และให้เขียน Scratch ด้วย NumPy ได้ ผลของ ReGLU อยู่ในหัวข้อ 4.7

4.4.2 NAM + log1p + Huber (regression)



ภาพที่ **4.5** architecture ของ NAM

การทำงานทีละขั้น:

1. **Subnet** ต่อฟีเจอร์**:** แต่ละฟีเจอร์ x_j ผ่าน neural network เล็กของตัวเอง f_j (Linear 1→64, ReLU, Linear 64→1)
2. ไม่มี subnet ใดเห็นฟีเจอร์อื่น
3. รวมด้วยการบวก**:** ẑ = b0 + Σ f_j(x_j)
4. แยกอ่านอิทธิพลของแต่ละฟีเจอร์ได้ตรง ๆ (glass-box)
5. เป้าหมายที่แปลงแล้ว**:** ฝึกกับ z-score ของ log1p(Income)
6. เหตุผล: รายได้เบ้ขวา (สูงสุด $162,397 เทียบค่าเฉลี่ย $51,959)

7. **Huber δ = 0.5:** ความคลาดเล็กคิดแบบกำลังสอง ความคลาดใหญ่คิดแบบเชิงเส้น
8. แถวรายได้สูงจึงดึงการฝึกน้อยกว่า MSE
9. คืนสเกล**:** Income = expm1(ẑ·σ + μ) เป็นหน่วยดอลลาร์
10. คำนวณตัวชี้วัดทุกตัวหลังคืนสเกล

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & |e| \le \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right) & |e| > \delta \end{cases}, \qquad \delta = 0.5$$

- พารามิเตอร์รวม 2,510 = 13 subnet × (64 + 64 + 64 + 1) + b0
- Adam (lr 1e-3) + cosine
- Weight decay 1e-4
- 100 epochs
- เก็บ checkpoint ที่ RMSE ต่ำสุด

3) ผลลัพธ์

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.6038**	**0.5197**	**0.0841**
ROC-AUC	0.8657	0.8251	0.0406
Precision	0.5217	0.5500	0.0283
Recall	0.7164	0.4925	0.2239
Accuracy	0.8594	0.8638	0.0044
BCE	0.4007	0.4278	0.0271

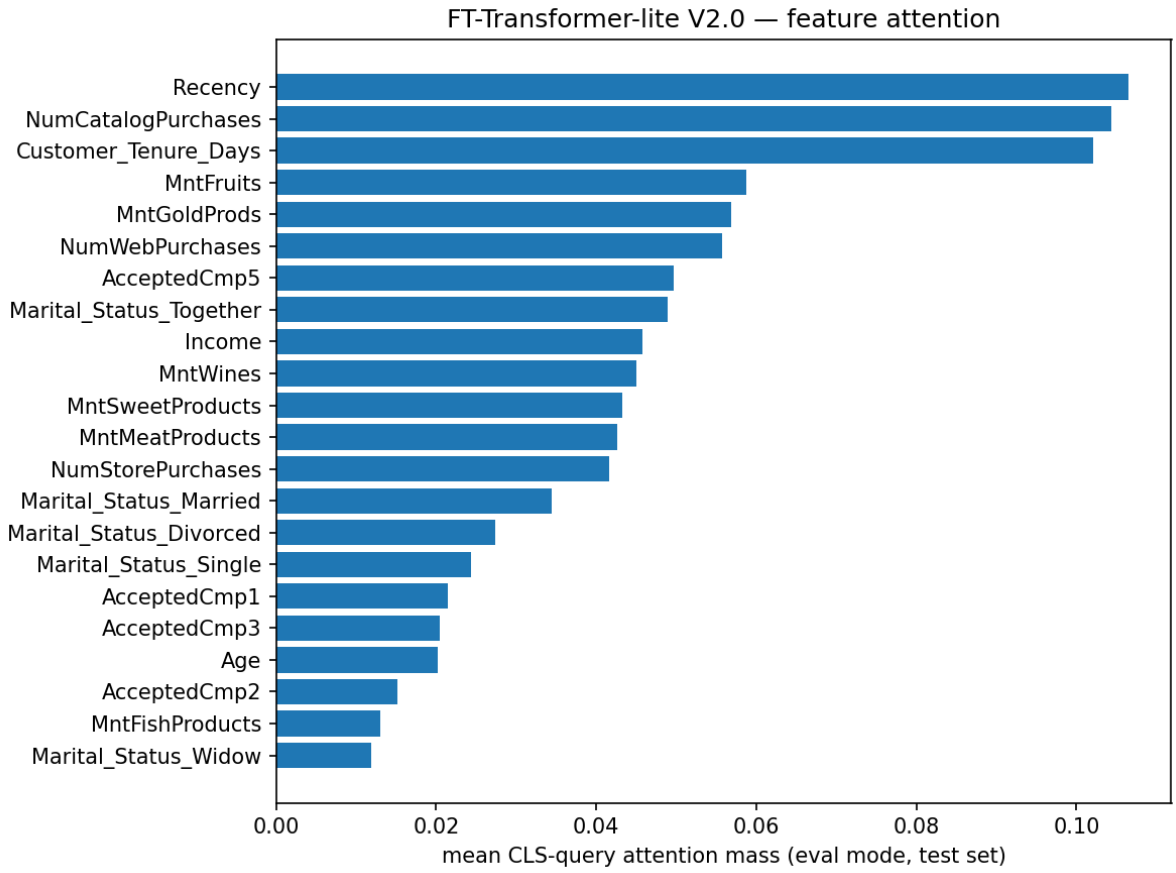
ตารางที่ **4.9** ผลลัพธ์ V2.0 (classify)

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7233**	**0.7286**	**0.0053**
RMSE ($)	11,463.5	11,353.3	1.0%
MAE ($)	7,186.2	7,064.1	1.7%

ตารางที่ **4.10** ผลลัพธ์ V2.0 (regression)

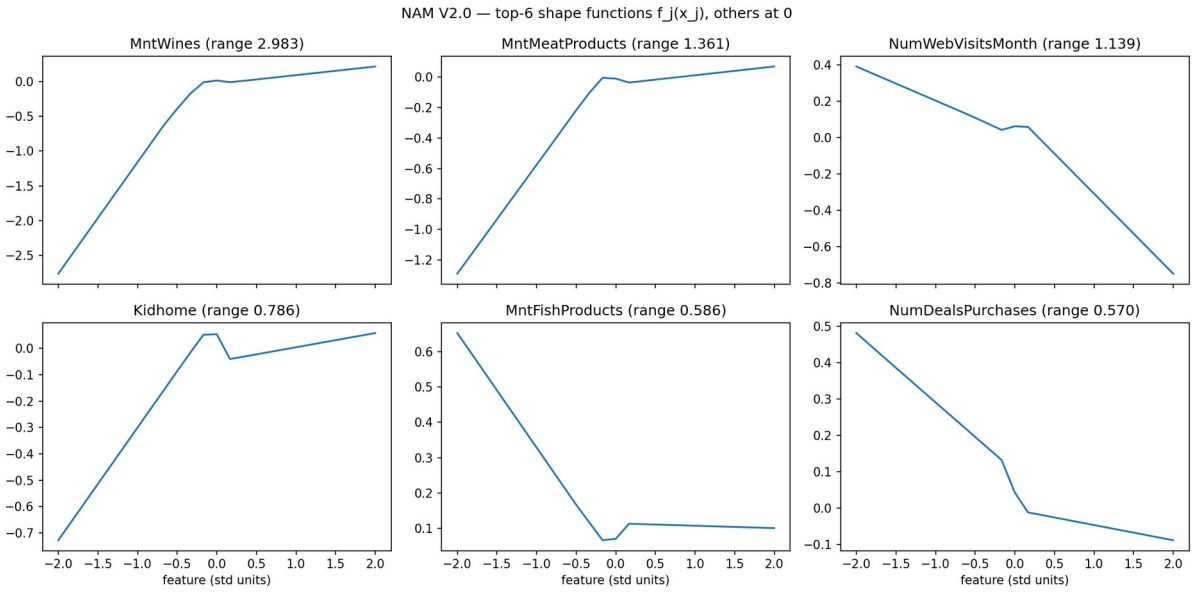
ตัววัด	V1.1	V2.0	เปลี่ยน
F1 · PyTorch	0.5455	0.6038	+0.0583
F1 · Scratch	0.5158	0.5197	+0.0039
R^2 · PyTorch	0.7774	0.7233	−0.0541
R^2 · Scratch	0.7595	0.7286	−0.0309

ตารางที่ **4.11** เปรียบเทียบผล V1.1 กับ V2.0



ภาพที่ **4.6** ค่า attention ของ FT-Transformer-lite

- 3 ฟีเจอร์บนสุดคือ `Recency` (0.107), `NumCatalogPurchases` (0.104) และ `Customer_Tenure_Days` (0.102)
- `Recency` และ `Customer_Tenure_Days` ตรงกับอันดับ 1–2 ของ RF (หัวข้อ 2.6) ไม่ใช่อันดับ bivariate ที่เป็น `AcceptedCmp1`, `AcceptedCmp2`, `AcceptedCmp3`, `AcceptedCmp4`, `AcceptedCmp5`



ภาพที่ **4.7** shape function ของ NAM

- MntWines มีช่วงผลใหญ่ที่สุด (2.983) รองลงมาคือ MntMeatProducts (1.361)
- MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds แต่ละตัวสูงขึ้น → รายได้ทำนายสูงขึ้น แต่กราฟจะชันที่ฝั่งค่าต่ำและแบนลงทางฝั่งสูง
- NumWebVisitsMonth และ NumDealsPurchases เป็นเส้นลดลง ตรงกับความสัมพันธ์เชิงลบใน EDA

4) ทำไมจึงเป็นเช่นนั้น

- classify: F1 ของ Library สูงกว่า V1.1 อยู่ 0.058 แต่ค่านี้ได้จาก checkpoint ที่เลือกบนชุดทดสอบ (กล่องข้างบน) จึงยังไม่ใช่หลักฐานว่า attention ชนะ MLP
- Scratch ใน classify แทบไม่ขยับ (+0.004)
- ความต่างระหว่างสองฝั่ง 0.084 เกินเกณฑ์ 0.05 เป็นครั้งแรกที่ parity ไม่ผ่าน เพราะข้อมูลมีน้อย (ฝึก 1,788 แถว ลูกค้าที่ตอบรับในชุดทดสอบแค่ 67 แถว) ทำให้ผลแกว่งง่าย แม้ BCE ของสองฝั่งจะต่างกันแค่ 0.027
- regression: NAM แพ้ V1.1 ทั้งสองฝั่ง (R^2 –0.054 และ –0.031) เพราะ NAM ดูแต่ละฟีเจอร์แยกกันแล้วค่อยบวกผล จึงใช้ข้อมูลจาก MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds พร้อมกัน ไม่ได้ ส่วน MLP หลาย Layer ใช้ได้

- ถ้าตัด 9 แถวที่ Income สูงสุดในชุดทดสอบออก NAM จะดีขึ้นมาก (RMSE $8,070, R^2 0.835) แสดงว่า NAM พลาดส่วนใหญ่ที่ลูกค้ารายได้สูงมาก ตัวเลขนี้ใช้ดูสาเหตุเท่านั้น ไม่ใช่คะแนนหลัก

หมายเหตุจากรอบที่สอง**:** เมื่อเลือก epoch จาก inner fold แทนชุดทดสอบ FT-Transformer-lite บน split เดียวกันได้ F1 0.497 (ไม่ใช่ 0.604) และ NAM ได้ R^2 0.719 (หัวข้อ 4.7)

5) คำถามถัดไป

- V1.1 ยังเป็นค่าอ้างอิงที่สูงที่สุดใน regression และคะแนน classify ของ V2.0 ไวต่อการเลือก checkpoint
- คำถามใหม่: ข้อมูลนี้ทำคะแนนได้สูงสุดเท่าไร โดยใช้ gradient boosting เป็นตัวเทียบ → V3.0

4.5 V3.0 — Gradient Boosting (LightGBM)

1) คำถาม**:** neural network อาจไม่ใช่วิธีที่เหมาะกับข้อมูลนี้ ลองใช้ gradient boosting แทนดีไหม

- ชุดข้อมูล ฟีเจอร์ (18 ตัวใน classify, 13 ตัวใน regression) และ split เหมือน V1.1 ทุกประการ ชุดทดสอบ 448 แถว (classify) และ 443 แถว (regression) เหมือนเดิม

2) architecture และกลไก

1. เริ่มจากค่าคงที่ F0
2. ทุกรอบคำนวณ gradient g ของ loss ต่อแถว
3. ทุกรอบคำนวณ Hessian h ของ loss ต่อแถว
4. สร้างต้นไม้ CART ลึกไม่เกิน 5 ชั้น (ใบละอย่างน้อย 20 แถว)
5. เลือกจุดแบ่งที่ให้ gain สูงสุด
6. คำนวณค่าใบไม้แบบ Newton step
7. บวกค่าใบไม้เข้าคำทำนายสะสม คูณ learning rate 0.05
8. วนกลับไปข้อ 2

$$\text{gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right], \qquad w_{\text{leaf}} = -\frac{G}{H+\lambda}$$

- classify: F0 = logit ของอัตราบวก ≈ −1.74
- classify: g = w·(p − y)
- classify: h = w·p(1 − p)
- classify: w = `scale_pos_weight` (5.697) สำหรับลูกค้าที่ตอบรับ และ 1 สำหรับลูกค้าที่ไม่ตอบรับ
- regression: F0 = ค่าเฉลี่ยของ `Income`
- regression: g = ค่าทำนาย − y, h = 1 (กลายเป็นการฟิตค่าคลาดเคลื่อนผ่านกลไกเดียวกัน)
- regression: เทรนบน `Income` ดิบ ไม่ทำ scaling (ต้นไม้ไม่ไวต่อสเกล)
- Library (LightGBM): `learning_rate` 0.05
- Library (LightGBM): `num_leaves` 31
- Library (LightGBM): subsample 0.8
- Library (LightGBM): colsample 0.8
- Library (LightGBM): `reg_lambda` 1.0
- Scratch: ต้นไม้ depth-wise ลึก 5 (แทน leaf-wise ของ LightGBM)
- Scratch: min leaf 20
- Scratch: พารามิเตอร์อื่นเหมือน Library
- Early stopping ทำบน 10% ที่เอาออกจากชุด **train** (patience 50)
- ไม่ใช้ชุดทดสอบ ซึ่งต่างจาก V2.0 ที่เลือก checkpoint บนชุดทดสอบ
- Sanity check: ต้นไม้ลึก 2 ตรงกับ `sklearn.tree.DecisionTreeRegressor` (ต่างสูงสุด 4×10^{-16})
- Sanity check: ตัวอย่าง boosting 3 แถว 2 รอบที่คำนวณมือได้ผลเท่ากัน
- Sanity check: loss ลดลงทุกรอบใน 20 รอบแรก

3) ผลลัพธ์

ตัววัด	LightGBM	Scratch	ส่วนต่าง
F1	**0.6014**	**0.0000**	**0.6014**
ROC-AUC	0.8898	0.8310	0.0588
Precision	0.5658	0.0000	0.5658
Recall	0.6418	0.0000	0.6418
Accuracy	0.8728	0.8504	0.0224
BCE	0.2967	0.3757	0.0790

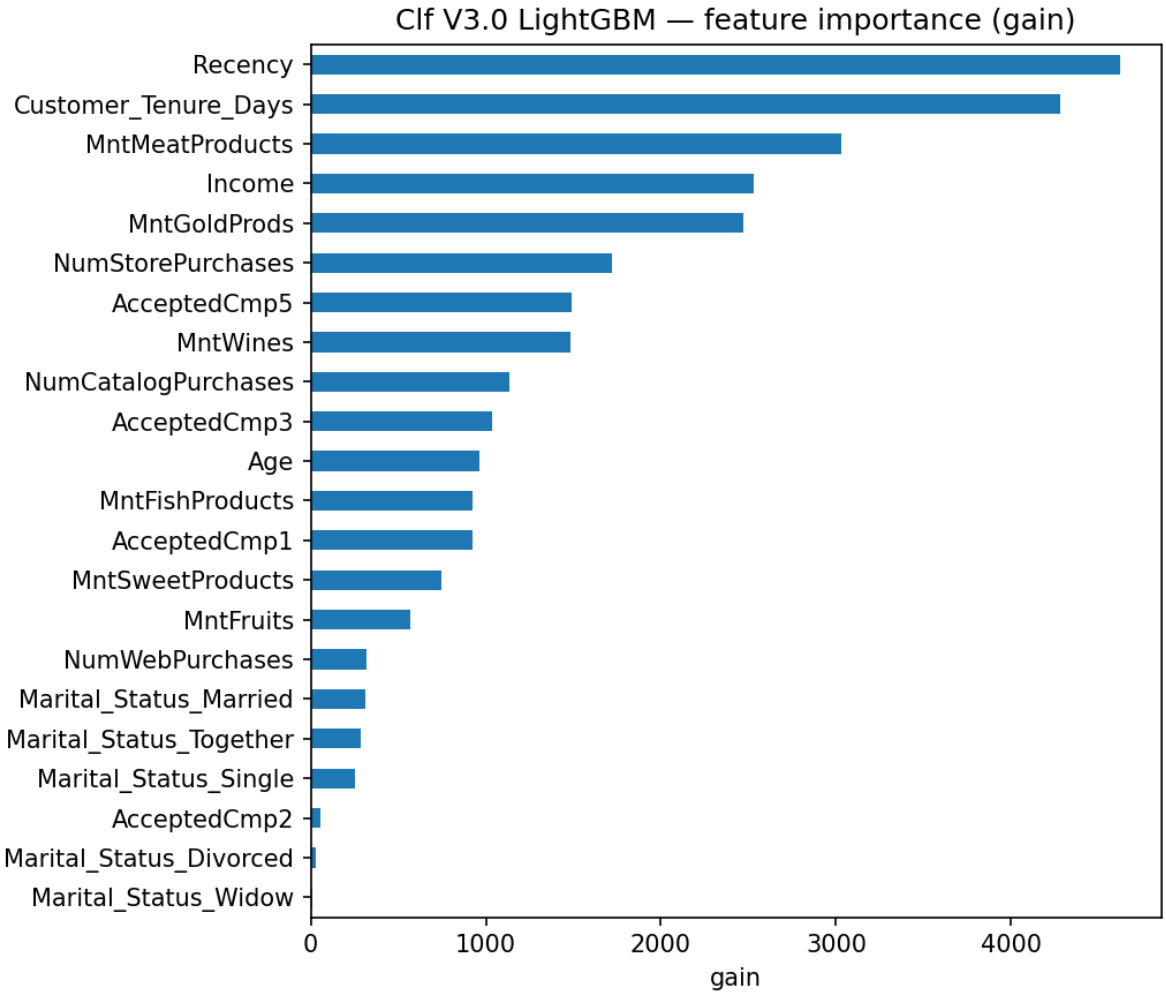
ตารางที่ **4.12** ผลลัพธ์ V3.0 (classify)

ตัววัด	LightGBM	Scratch	ส่วนต่าง
R^2	**0.8212**	**0.8213**	**0.0001**
RMSE ($)	9,216.7	9,212.9	0.04%
MAE ($)	5,758.4	5,904.9	2.5%

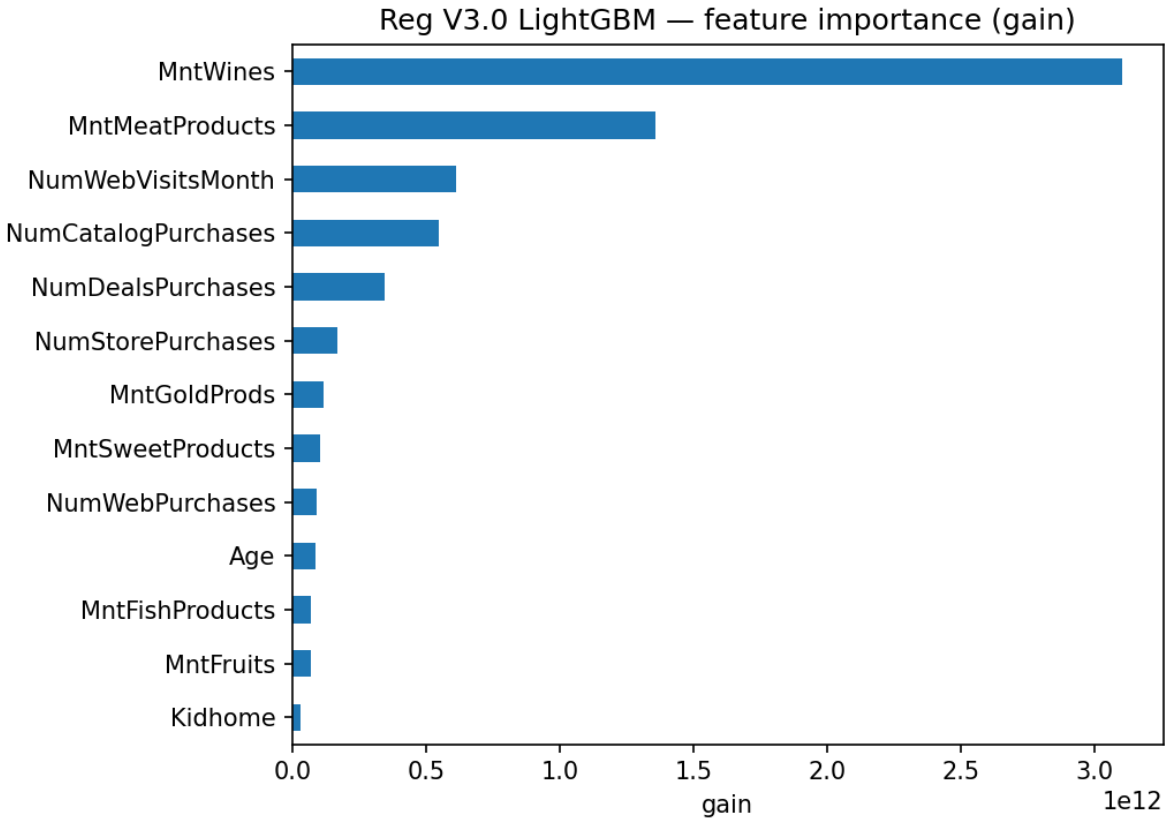
ตารางที่ **4.13** ผลลัพธ์ V3.0 (regression)

ตัววัด	V2.0	V3.0	เปลี่ยน
F1 · Library	0.6038	0.6014	−0.0024
F1 · Scratch	0.5197	0.0000	−0.5197
R^2 · Library	0.7233	0.8212	+0.0979
R^2 · Scratch	0.7286	0.8213	+0.0927

ตารางที่ **4.14** เปรียบเทียบผล V2.0 กับ V3.0



ภาพที่ **4.8** feature importance ของ LightGBM (classify)



ภาพที่ **4.9** feature importance ของ LightGBM (regression)

4) ทำไมจึงเป็นเช่นนั้น

- regression: R^2 0.821 สูงกว่า V1.1 (0.777) อยู่ 0.044 และสองฝั่งได้ผลเท่ากัน (ต่าง 0.0001) ซึ่งเป็นสัมพันธ์ชัดที่สุดในโครงงาน แสดงว่าข้อมูลนี้เหมาะกับโมเดลแบบต้นไม้ (tree-based) มากกว่า neural network
- ฟีเจอร์ regression อันดับบนคือ `MntWines` และ `MntMeatProducts` ตามด้วย `NumWebVisitsMonth`, `NumCatalogPurchases`, `NumDealsPurchases` สอดคล้องกับเรื่อง “`MntWines`, `MntFruits`, `MntMeatProducts`, `MntFishProducts`, `MntSweetProducts`, `MntGoldProds` เป็นตัวแทนของ `Income`” ใน EDA
- classify: F1 ของ LightGBM (0.601) สูงกว่า V1.1 (0.546) อยู่ 0.056 และใกล้ V2.0 Library มาก (0.604) ทั้ง AUC 0.890 สูงที่สุดในทุกโมเดล
- `Recency` และ `Customer_Tenure_Days` อยู่อันดับ 1–2 เหมือน RF และ attention ของ FT-Transformer-lite สามวิธีชี้ไปทางเดียวกัน: สัญญาณจาก

`Recency`/`Customer_Tenure_Days` แข็งกว่าสัญญาณจาก `AcceptedCmp1`, `AcceptedCmp2`, `AcceptedCmp3`, `AcceptedCmp4`, `AcceptedCmp5` โดยตรง

- **Parity classify fail:** Scratch ได้ F1 = 0 เพราะ early stopping บน BCE หยุดที่รอบ 8 ซึ่งความน่าจะเป็นทำนายทุกแถวให้อยู่ในช่วง 0.12–0.19 ไม่เคยเกิน threshold 0.5 แม้ ROC-AUC (0.831) ยังใกล้ LightGBM
- กลไกเกิดจากความไม่สมดุล: ค่าเริ่มต้น F0 ≈ −1.74 ทำให้ BCE ดีอยู่แล้วเมื่อทำนายใกล้อัตราฐาน ในขณะที่ learning rate เล็ก (0.05) ยังไม่ทันยกคะแนนแถวใดขึ้นไปเหนือศูนย์
- การตัดแถวเดียวออกจากชุด train ย้ายจุด BCE ต่ำสุดจากรอบที่ 254 ไปรอบที่ 8 แสดงว่าจุดหยุดไวต่อการแบ่งข้อมูลมาก

5) คำถามถัดไป

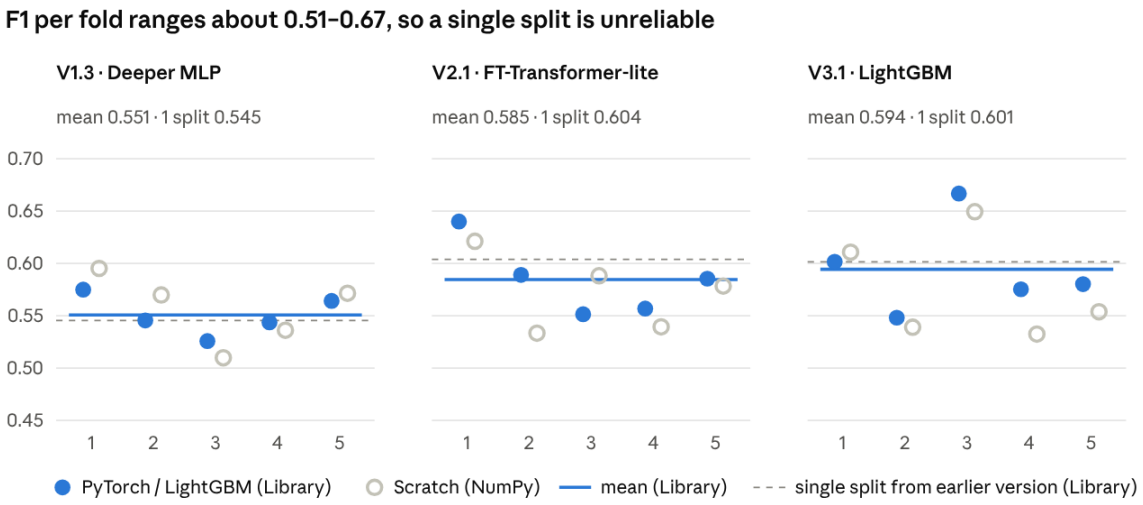
- คะแนนจาก split เดียวไม่น่าเชื่อพอ เพราะชุดทดสอบมีลูกค้าที่ตอบรับแค่ 67 แถว → รัน 5-fold ให้ในทุกครอบครัวโมเดล (V1.3, V2.1, V3.1)

4.6 V1.3, V2.1, V3.1 — ตรวจซ้ำด้วย **5-fold cross-validation**

1) คำถาม**:** ผลจาก split 80/20 เดียวเป็นตัวแทนที่ดีหรือเป็นแค่การสุ่มที่โชคดี/ร้าย โดยเฉพาะจุดที่น่าสงสัยคือ F1 0.6038 ของ V2.0 และ F1 = 0 ของ V3.0 scratch

2) วิธีทำ**:** ใช้ architecture และ hyper-parameter เดิมคัดลอกจาก V1.1, V2.0, V3.0 ไม่แก้ แล้วห่อด้วย 5-fold (รายละเอียดในหัวข้อ 3.6)

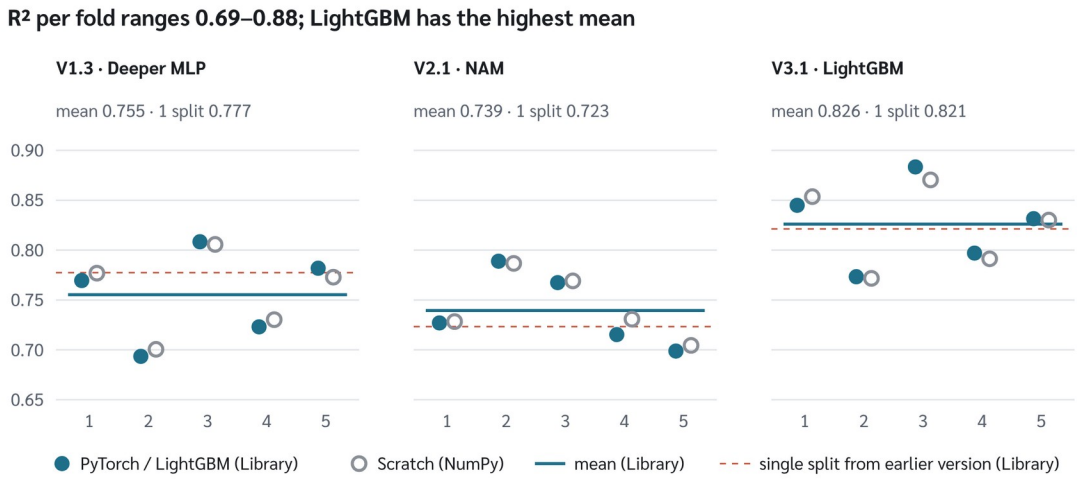
3) ผลลัพธ์



ภาพที่ **4.10** F1 ของแต่ละ fold (classify)

เวอร์ชัน	PyTorch / LightGBM (mean ± SD)	Scratch (mean ± SD)	split เดียว (Library)
V1.3 MLP หลาย Layer	0.5502 ± 0.0200	0.5565 ± 0.0335	0.5455
V2.1 FT-Transformer-lite	0.5723 ± 0.0380	0.5720 ± 0.0363	0.6038
V3.1 LightGBM	0.5944 ± 0.0447	0.5771 ± 0.0509	0.6014

ตารางที่ **4.15** ค่าเฉลี่ย F1 จาก 5-fold (classify)



ภาพที่ **4.11** R^2 ของแต่ละ fold (regression)

เวอร์ชัน	PyTorch / LightGBM (mean ± SD)	Scratch (mean ± SD)	split เดียว (Library)
V1.3 MLP หลาย Layer	0.7553 ± 0.0463	0.7573 ± 0.0416	0.7774
V2.1 NAM	0.7395 ± 0.0375	0.7439 ± 0.0333	0.7233
V3.1 LightGBM	0.8260 ± 0.0427	0.8234 ± 0.0414	0.8212

ตารางที่ **4.16** ค่าเฉลี่ย R^2 จาก 5-fold (regression)

4) ทำไมจึงเป็นเช่นนั้น

- ความแกว่งระหว่าง fold (SD ราว 0.02–0.05 ใน F1 และ R^2) ใหญกว่าส่วนต่างระหว่าง architecture หลายคู่ของค่าเฉลี่ย
- classify: ค่าที่ V2.0 รายงาน (0.6038) สูงกว่าค่าเฉลี่ย 5-fold (0.5723) จึงเป็น split ที่โชคดี ส่วน V3.0 (0.6014) และ V1.1 (0.5455) อยู่ใกล้ค่าเฉลี่ย
- Parity fail ของ V2.0 ไม่เกิดซ้ำ: ส่วนต่างค่าเฉลี่ย Library–scratch เหลือ 0.0003 และทุก fold ต่างกันไม่เกิน 0.051 ซึ่งยืนยันว่าความต่าง 0.084 ใน V2.0 มาจากการเลือก epoch ที่ไม่เท่ากัน ไม่ใช่การคำนวณที่ต่างกัน
- F1 = 0 ของ V3.0 scratch ไม่เกิดใน fold ใดเลย (best_iter 235–369 ทุก fold) เป็นผลจากจุดหยุด early-stopping ของ split เดียว ไม่ใช่ข้อบกพร่องอย่างเป็นระบบของวิธี

- ถ้าเทียบทีละ fold (ทุกเวอร์ชันใช้ seed 42 และวิธีแบ่งเดียวกัน) FT-Transformer-lite ได้ F1 สูงกว่า MLP หลาย Layer 4 จาก 5 fold (–0.007 ถึง +0.054, เฉลี่ย +0.022) แต่ต้องระวัง: V2.1 เลือก checkpoint บน fold ที่ใช้รายงาน ส่วน V1.3 ฝึกคงที่ 100 epochs gap บางส่วนจึงมาจากวิธีเลือก checkpoint
- LightGBM ไม่ได้ชนะ FT-Transformer-lite อย่างสม่ำเสมอใน classify (สูงกว่า 3 จาก 5 fold ผลต่างเฉลี่ย 0.022 เทียบกับ SD 0.04) จึงบอกได้แค่ว่าใกล้เคียงกัน
- regression: LightGBM ได้ R^2 สูงกว่า MLP หลาย Layer ทุก fold (+0.05 ถึง +0.08) และสูงกว่า NAM 4 จาก 5 fold (NAM ชนะใน fold 2) ผลสรุปก่อนหน้า “regression เหมาะกับ GBDT ที่สุด” จึงยังเป็นจริงบน 5-fold
- NAM แพ้ MLP หลาย Layer ที่ค่าเฉลี่ย 0.016 แต่ไม่ทุก fold (ชนะเฉพาะ fold 2) ราคาของความอธิบายได้จึงผันแปรตาม fold ไม่ใช่จำนวนคงที่

5) คำถามถัดไป

- ไม่มี validation set แยกในทุกเวอร์ชันที่เลือก checkpoint บน fold/test จึงควรบันทึกเป็นข้อจำกัด (บทที่ 5) และใช้ค่า mean ± SD เป็นตัวเลขหลักในการเปรียบเทียบ

4.7 V4 — ReGLU: ทำไม FT-Transformer ใช้ และช่วยเราหรือไม่

1) คำถาม**:** FT-Transformer ต้นฉบับ [1] ใช้ ReGLU ใน FFN แทน ReLU เพราะอะไร และถ้าใส่ใน FT-Transformer-lite กับ NAM ของเราจะดีขึ้นหรือไม่

2) architecture

- ReGLU แบ่งผลของ Linear ชั้นแรกเป็นสองครึ่ง: value และ gate
- ผลลัพธ์ = value × ReLU(gate) ทำให้แต่ละหน่วยซ่อนถูก "เปิดหรือปิด" ได้ตามอินพุต
- Shazeer (2020) [16] พบว่า FFN แบบ GLU ให้ Transformer ด้านภาษาดีกว่า ReLU ที่พารามิเตอร์เท่ากัน FT-Transformer จึงนำมาใช้ และตั้ง hidden = $4/3 \cdot d$ เพื่อคุมจำนวนพารามิเตอร์
- ทดสอบ 3 รุ่น: ReLU (เดิม), ReGLU ขนาด hidden เท่าเดิม (พารามิเตอร์มากขึ้น) และ ReGLU ที่ลด hidden ให้พารามิเตอร์เท่า ReLU
- NAM ใช้ ReGLU ใน subnet ของทุกฟีเจอร์ ด้วยหลักเดียวกัน

- ฝึกตามกติกาหัวข้อ 3.8 (epoch เลือกจาก inner 3-fold)

$$\mathrm{ReGLU}(x) = (xW_v + b_v) \odot \max(0,\, xW_g + b_g), \qquad \mathrm{FFN}(x) = \mathrm{ReGLU}(x)\, W_2 + b_2$$

รุ่น	hidden (classify / regression)	พารามิเตอร์ classify	พารามิเตอร์ regression
ReLU (เดิม)	64 / 64	18,625	2,510
ReGLU, hidden เท่าเดิม	64 / 64	22,849	4,174
ReGLU, พารามิเตอร์เท่ากัน	43 / 38	18,733	2,484

ตารางที่ **4.17** จำนวนพารามิเตอร์ของ FFN แต่ละแบบ

3) ผลลัพธ์

รุ่น	**F1** (mean ± SD)	ต่างจาก ReLU	95% CI
ReLU (เดิม)	**0.547 ± 0.037**	–	–
ReGLU, hidden เท่าเดิม	**0.545 ± 0.050**	–0.003	–0.023 ถึง +0.018
ReGLU, พารามิเตอร์เท่ากัน	**0.521 ± 0.060**	–0.027	–0.052 ถึง –0.003

ตารางที่ **4.18** ผลลัพธ์ V4 (classify)

รุ่น	R^2 (mean ± SD)	ต่างจาก ReLU	95% CI
ReLU (เดิม)	**0.735 ± 0.065**	–	–
ReGLU, hidden เท่าเดิม	**0.724 ± 0.099**	–0.011	–0.048 ถึง +0.023
ReGLU, พารามิเตอร์เท่ากัน	**0.726 ± 0.094**	–0.010	–0.040 ถึง +0.021

ตารางที่ **4.19** ผลลัพธ์ V4 (regression)

- ไม่มีรุ่น ReGLU ใดดีกว่า ReLU ในทั้งสองงาน (corrected t-test $p \geq 0.37$ ทุกคู่)
- บน split เดียว NAM + ReGLU ดูเหมือนดีขึ้น (R^2 0.719 → 0.762) แต่หายไปเมื่อดู 15 folds
- SD ของ ReGLU สูงกว่า ReLU ทุกรุ่น (เช่น 0.099 เทียบ 0.065 ใน regression)
- Scratch: forward, loss และ gradient ตรงกับ PyTorch บนน้ำหนักเดียวกัน (ต่างไม่เกิน 10^{-13}) และจำนวนพารามิเตอร์ตรงกัน

4) ทำไมจึงเป็นเช่นนั้น

- ประโยชน์ของ gate พบในโมเดลภาษาที่ฝึกด้วยข้อมูลมหาศาล ส่วนเรามีแถวฝึกราว 1,800 แถวต่อ fold และลูกค้าที่ตอบรับราว 270 แถว
- gate เพิ่มความยืดหยุ่น เมื่อข้อมูลน้อย ความยืดหยุ่นนี้กลายเป็นความแกว่ง (SD สูงขึ้น) มากกว่าสัญญาณ
- รุ่นพารามิเตอร์เท่ากันต้องลด hidden จาก 64 เหลือ 43 จึงเสียความกว้างของชั้นไป
- ใน NAM แต่ละ subnet รับค่าเดียว การ gate ฟังก์ชันของตัวแปรเดียวเพิ่มสิ่งที่ ReLU ทำได้อยู่แล้วไม่มาก

5) คำถามถัดไป

- คง ReLU ไว้เป็นค่าหลัก แล้วถามว่าการใส่ interaction ที่พบในหัวข้อ 2.9 ช่วยได้หรือไม่ → V5

4.8 V5 — ใช้ Feature Interaction ในโมเดล

1) คำถาม**:** คู่ฟีเจอร์ที่ส่งผลร่วมกัน (หัวข้อ 2.9) ช่วยให้โมเดลดีขึ้นหรือไม่ และช่วย extracurricular มากกว่าต้นไม้หรือไม่

2) architecture

- classify: เพิ่ม 3 ฟีเจอร์ที่เคยตัด (`Teenhome`, `NumDealsPurchases`, `NumWebVisitsMonth`) แล้วเพิ่มผลคูณของ 5 คู่ที่แรงที่สุดเป็นฟีเจอร์ใหม่ (22 → 25 → 30 มิติ)
- ผลคูณคำนวณจากค่าที่ standardize แล้ว และ fit scaler บนชุด train เท่านั้น
- regression: เพิ่ม `Teenhome` (13 → 14) แล้วเพิ่ม **pair subnet** ให้ NAM กลายเป็น NA²M (คล้าย GA²M [17])
- pair subnet รับ 2 ฟีเจอร์พร้อมกัน (Linear 2→64, ReLU, Linear 64→1) ส่วนที่เหลือยังบวกกันเหมือนเดิม
- ทดสอบ 2 ชุดคู่: 4 คู่ที่แรงที่สุดใน 13 ฟีเจอร์เดิม และ 5 คู่ที่มี `Teenhome` (`NumCatalogPurchases`, `NumWebVisitsMonth`, `MntWines`, `MntMeatProducts`, `NumStorePurchases`)
- LightGBM ได้ชุดฟีเจอร์เดียวกันทุกชุด เพื่อเทียบอย่างเป็นธรรม (ต้นไม้ไม่ต้องใช้ pair subnet)

$$\hat{z} = b_0 + \sum_j f_j(x_j) + \sum_{(i,k)\in\mathcal{P}} g_{ik}(x_i, x_k)$$

3) ผลลัพธ์

ชุดฟีเจอร์	**F1** FT-Transformer-lite	**F1** LightGBM	ต่างจากชุด 22 มิติ (FT / LightGBM)
22 มิติ (เดิม)	**0.556 ± 0.042**	**0.581 ± 0.050**	–
+ 3 ฟีเจอร์ (25)	**0.553 ± 0.044**	**0.585 ± 0.050**	–0.003 / +0.004
+ 3 ฟีเจอร์ + 5 คู่ (30)	**0.558 ± 0.042**	**0.604 ± 0.044**	+0.002 / **+0.023**

ตารางที่ **4.20** ผลลัพธ์ V5 (classify)

ชุดฟีเจอร์	**R^2** NAM / NA²M	**R^2** LightGBM	ต่างจากชุด 13 ฟีเจอร์ (NAM / LightGBM)
13 ฟีเจอร์ (เดิม)	**0.735 ± 0.065**	**0.813 ± 0.064**	–
13 ฟีเจอร์ + 4 pair subnet	**0.725 ± 0.066**	–	–0.011 / –
+ Teenhome (14)	**0.748 ± 0.068**	**0.826 ± 0.054**	+0.012 / +0.013
+ Teenhome + 5 pair subnet	**0.757 ± 0.074**	–	**+0.021** / –

ตารางที่ **4.21** ผลลัพธ์ V5 (regression)

- classify: เพิ่ม 3 ฟีเจอร์ที่มี interaction สูง (Teenhome, NumDealsPurchases, NumWebVisitsMonth) และฟีเจอร์คู่ 5 ตัว ซึ่งคือผลคูณของคู่ที่มี interaction สูงสุดจาก EDA (เช่น Teenhome × NumWebVisitsMonth) ทำให้ F1 ของ LightGBM เพิ่มขึ้น +0.023 เทียบกับชุด 22 ฟีเจอร์เดิม (95% CI +0.007 ถึง +0.039, Wilcoxon p = 0.030) แต่ F1 ของ FT-Transformer-lite แทบไม่เปลี่ยน (+0.002, CI –0.018 ถึง +0.023)

- regression: เพิ่ม `Teenhome` ทำให้ R^2 ของ LightGBM และ NAM เพิ่มขึ้นใกล้เคียงกัน (+0.012 / +0.013) และ pair subnet ที่มี `Teenhome` ช่วย NAM เพิ่มอีกเล็กน้อย
- การเพิ่ม pair subnet 4 คู่ให้ 13 ฟีเจอร์เดิมทำให้ R^2 ของ NAM ลดลง (–0.011, CI –0.017 ถึง –0.005)
- ผลต่างเหล่านี้ผ่านการทดสอบ Wilcoxon ($p \leq 0.041$) แต่ไม่ผ่าน corrected t-test ($p \geq 0.086$) จึงถือเป็นหลักฐานระดับอ่อน
- เวอร์ชัน Scratch ของ FT-Transformer-lite (30 มิติ) และ NA^2M ให้ผลตรงกับ PyTorch บนน้ำหนักชุดเดียวกัน (ต่างไม่เกิน 10^{-14})

4) ทำไมจึงเป็นเช่นนั้น

- FT-Transformer-lite ผสมฟีเจอร์ผ่าน attention ได้อยู่แล้ว ผลคูณที่เติมให้จึงไม่ใช่ข้อมูลใหม่สำหรับมัน
- ต้นไม้แบ่งทีละแกนและต้องใช้หลายชั้นกว่าจะจับผลคูณได้ การให้ผลคูณตรง ๆ จึงช่วย LightGBM มากกว่า
- NAM เดิมผสมฟีเจอร์ไม่ได้เลย pair subnet ที่ตรงกับ interaction จริง (`Teenhome`) จึงช่วย ส่วนคู่ที่อ่อนกว่าเพิ่มพารามิเตอร์โดยไม่เพิ่มสัญญาณ
- คู่ถูกเลือกจาก EDA ที่ใช้ทุกแถว ผลบวกในตารางจึงอาจสูงเกินจริงเล็กน้อย

5) คำถามถัดไป

- ทั้ง ReGLU และ interaction ยังไม่ทำให้ extracurricular แซง LightGBM ได้ → V6 เทียบกับทุกโมเดลในหลักสูตร

4.9 V6 — โมเดลในหลักสูตรทั้งหมดบน **fold** เดียวกัน

1) คำถาม**:** เมื่อให้โมเดลในหลักสูตรทุกตัว (รวม XGBoost และ CatBoost) ได้ปรับ hyper-parameter และทดสอบบน fold เดียวกัน extracurricular ยังชนะอยู่หรือไม่

2) วิธีทำ

- 4 ชุดโน้ตบุ๊กต่องาน: scikit-learn, XGBoost [5], LightGBM [3], CatBoost [18]
- classify 19 โมเดล และ regression 14 โมเดล
- ทุกโมเดลผ่าน Optuna [19] 30 trials บน inner 3-fold ทุก outer fold แยกกัน (nested cross-validation)

- k-Means และ agglomerative ใช้อัตราตอบรับของคลัสเตอร์เป็นคะแนนทำนาย

3) ผลลัพธ์

กลุ่ม	โมเดล	**F1** (mean ± SD)	อันดับ (จาก 28)
Boosting	LightGBM	**0.590 ± 0.048**	2
Stacking	RF + GB + SVM	**0.574 ± 0.054**	8
Bagging	Random Forest	**0.565 ± 0.051**	10
SVM	SVM (RBF)	**0.558 ± 0.050**	12
extracurricular	FT-Transformer-lite (+ ฟีเจอร์คู่)	**0.558 ± 0.042**	13
Neural network	MLP	**0.556 ± 0.065**	14
Linear	Logistic regression	**0.540 ± 0.058**	21
Distance	kNN	**0.536 ± 0.052**	22
Probabilistic	Naïve Bayes	**0.478 ± 0.038**	24
Clustering	k-Means	**0.472 ± 0.039**	25
Perceptron	SLP	**0.456 ± 0.066**	27
Tree	Decision Tree	**0.444 ± 0.074**	28

ตารางที่ **4.22** ผลลัพธ์ V6 (classify)

กลุ่ม	โมเดล	**R^2** (mean ± SD)	อันดับ (จาก 21)
Boosting	CatBoost	**0.830 ± 0.070**	1
Bagging	ExtraTrees	**0.821 ± 0.069**	3

กลุ่ม	โมเดล	**R^2** (mean ± SD)	อันดับ (จาก 21)
SVM	SVR (RBF)	**0.789 ± 0.060**	9
Neural network	MLP	**0.769 ± 0.092**	10
Distance	kNN	**0.769 ± 0.059**	11
extracurricular	NA²M (+ Teenhome pair subnet)	**0.757 ± 0.074**	12
Regression ในหลักสูตร	Polynomial	**0.751 ± 0.069**	13
Regression ในหลักสูตร	Multiple	**0.733 ± 0.043**	17
Regression ในหลักสูตร	Linear (1 ฟีเจอร์)	**0.446 ± 0.064**	21

ตารางที่ **4.23** ผลลัพธ์ V6 (regression)

- ลำดับเต็มทุกโมเดลอยู่ในหัวข้อ 5.1
- boosting ทุกตัว (LightGBM, CatBoost, XGBoost, HistGradientBoosting) อยู่หัวตารางทั้งสองงาน ห่างกันไม่เกิน 0.012 (classify) และ 0.017 (regression)
- LightGBM ใน V6 (0.590) ใกล้ V3.1 (0.594) แม้จะใช้ Optuna และเลือก threshold ใหม่

4) ทำไมจึงเป็นเช่นนั้น

- ข้อมูลเป็นตารางขนาดเล็ก (ราว 2,200 แถว) ที่มีฟีเจอร์หลายชนิด ซึ่งเป็นสภาพที่งานวิจัยพบว่า tree boosting มักชนะ [13]
- โมเดลเชิงเส้น (Logistic, Multiple) ได้คะแนนห่างจากอันดับ 1 เพียง 0.064 F1 และ 0.097 R^2 สัญญาณส่วนใหญ่จึงเป็นแบบเรียบง่าย ส่วนที่เหลือเป็นส่วนที่ต้นไม้เก็บได้ดีกว่า
- สาเหตุที่ extracurricular แพ้มีหลักฐานใน V7 (หัวข้อ 4.10) และสรุปในหัวข้อ 5.3

5) คำถามถัดไป

- gap ระหว่าง extracurricular กับ boosting มาจากอะไร: interaction, รูปแบบการเรียนผลของแต่ละฟีเจอร์ หรือขนาดข้อมูล → V7

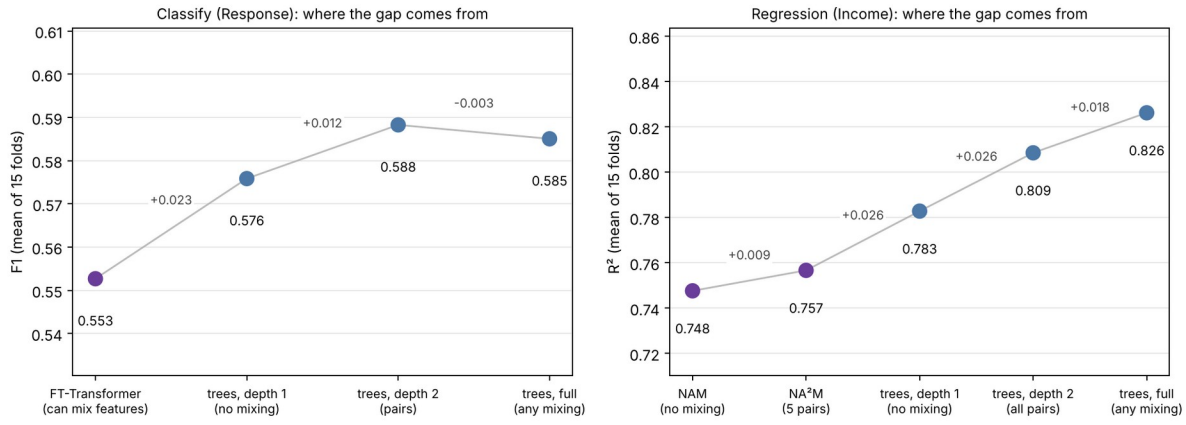
4.10 V7 — หาสาเหตุของ gap

1) คำถาม: gap ระหว่าง extracurricular กับ boosting มาจาก (ก) interaction ที่ต้นไม้จับได้ (ข) รูปแบบการเรียนผลของแต่ละฟีเจอร์ (ต้นไม้แบ่งค่าเป็นขั้น ส่วน neural network เรียนเป็นเส้นโค้งเรียบ) หรือ (ค) ข้อมูลน้อยเกินไป

2) วิธีทำ

- บันได **interaction:** ฝึก LightGBM ที่จำกัดความลึกต้นไม้ 1 ชั้น (บวกกันอย่างเดียว เหมือน NAM) และ 2 ชั้น (ผสมได้ทีละคู่ เหมือน NA²M) แล้วเทียบกับ LightGBM เต็ม
- ทุกขั้นผ่าน Optuna 30 trials บน 15 folds และชุดฟีเจอร์เดียวกับโมเดลฝั่ง extracurricular (25 มิติ / 14 ฟีเจอร์)
- **learning curve:** ฝึกด้วย 25%, 50%, 75%, 100% ของชุด train แต่ละ fold (5 folds ของ seed 42) โดยทั้งสองโมเดลได้แถวชุดเดียวกัน
- ภาพอธิบาย **NAM:** ฝึก NA²M ตัวสุดท้ายบน split 80/20 (R^2 0.764, MAE $6,394) แล้วอ่าน shape function ตรงจากโมเดล (ภาพในหัวข้อ 5.5)

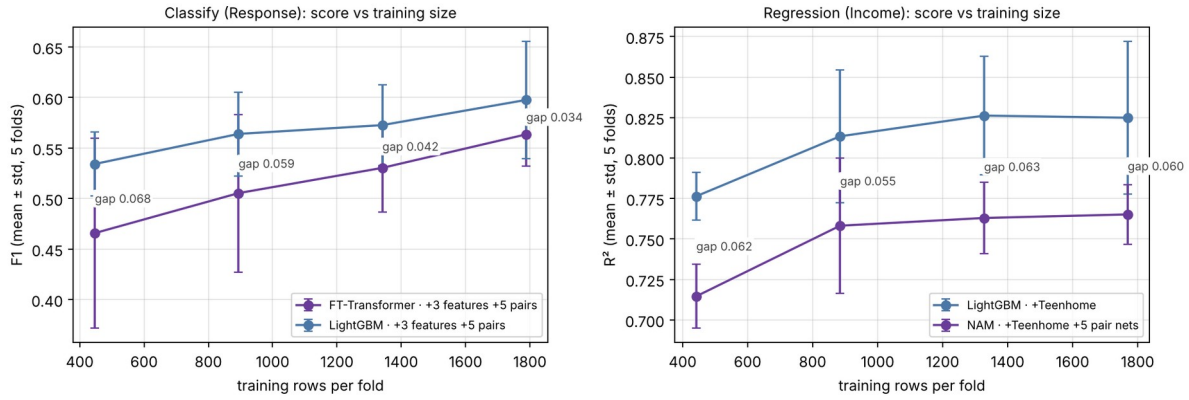
3) ผลลัพธ์



ภาพที่ **4.12** คะแนนตามระดับความลึกของ interaction

ขั้น	ผลต่าง	95% CI	Wilcoxon p
classify: FT-Transformer-lite → ต้นไม้ลึก 1	**+0.023**	+0.004 ถึง +0.043	0.095
classify: ลึก 1 → ลึก 2	**+0.012**	–0.007 ถึง +0.031	0.21
classify: ลึก 2 → เต็ม	**–0.003**	–0.021 ถึง +0.015	0.68
regression: NAM → ต้นไม้ลึก 1 (บวกกันทั้งคู่)	**+0.035**	+0.010 ถึง +0.058	0.030
regression: ลึก 1 → ลึก 2	**+0.026**	+0.013 ถึง +0.038	0.003
regression: ลึก 2 → เต็ม	**+0.018**	+0.011 ถึง +0.023	0.0009

ตารางที่ **4.24** ผลต่างของคะแนนในแต่ละระดับความลึกของ interaction



ภาพที่ **4.13** learning curve ของโมเดล

- classify: gap หดจาก 0.068 (447 แถว) เหลือ 0.034 (1,789 แถว) และ FT-Transformer-lite ยังชันขึ้นที่ปลายเส้น
- regression: gap คงที่ราว 0.06 ทุกขนาด และทั้งสองโมเดลแบนตัวหลัง 1,300 แถว

4) ทำไมจึงเป็นเช่นนั้น

- classify: ต้นไม้ที่ ผสมฟีเจอร์ไม่ได้เลย ยังชนะ FT-Transformer-lite ที่ผสมได้ สาเหตุหลักจึงไม่ใช่ interaction แต่เป็นรูปแบบการเรียนผลของแต่ละฟีเจอร์
- ต้นไม้ตัดที่เกณฑ์เดียวได้ทันที เหมาะกับฟีเจอร์ 0/1 และจำนวนนับ (เช่น `AcceptedCmp5`, `Recency`) neural network ต้องเรียนเส้นโค้งที่เรียบจากข้อมูลจำนวนมาก สอดคล้องกับ [13]
- classify: learning curve ยืนยันว่า FT-Transformer-lite ต้องการข้อมูลมากกว่า (ลูกค้าที่ตอบรับมีเพียงราว 270 แถวต่อ fold) gap จึงน่าจะแคบลงเมื่อมีข้อมูลมากขึ้น
- regression: gap 0.078 (NAM 0.748 → LightGBM 0.826) แบ่งได้เป็นส่วนที่มาจากรูปแบบการเรียนผลรายฟีเจอร์ 0.035 (45%) และ interaction ที่ NAM ไม่มี 0.044 (55%)
- NA²M เพิ่มได้เพียง 0.009 เพราะมีแค่ 5 คู่ ต้นไม้ลึก 2 ใช้ได้ทุกคู่ (91 คู่)
- regression: gap ไม่หดตามขนาดข้อมูล จึงเป็นเพราะ NAM จับ interaction ระหว่างฟีเจอร์ไม่ได้ ไม่ใช่เพราะข้อมูลน้อย

5) คำถามถัดไป

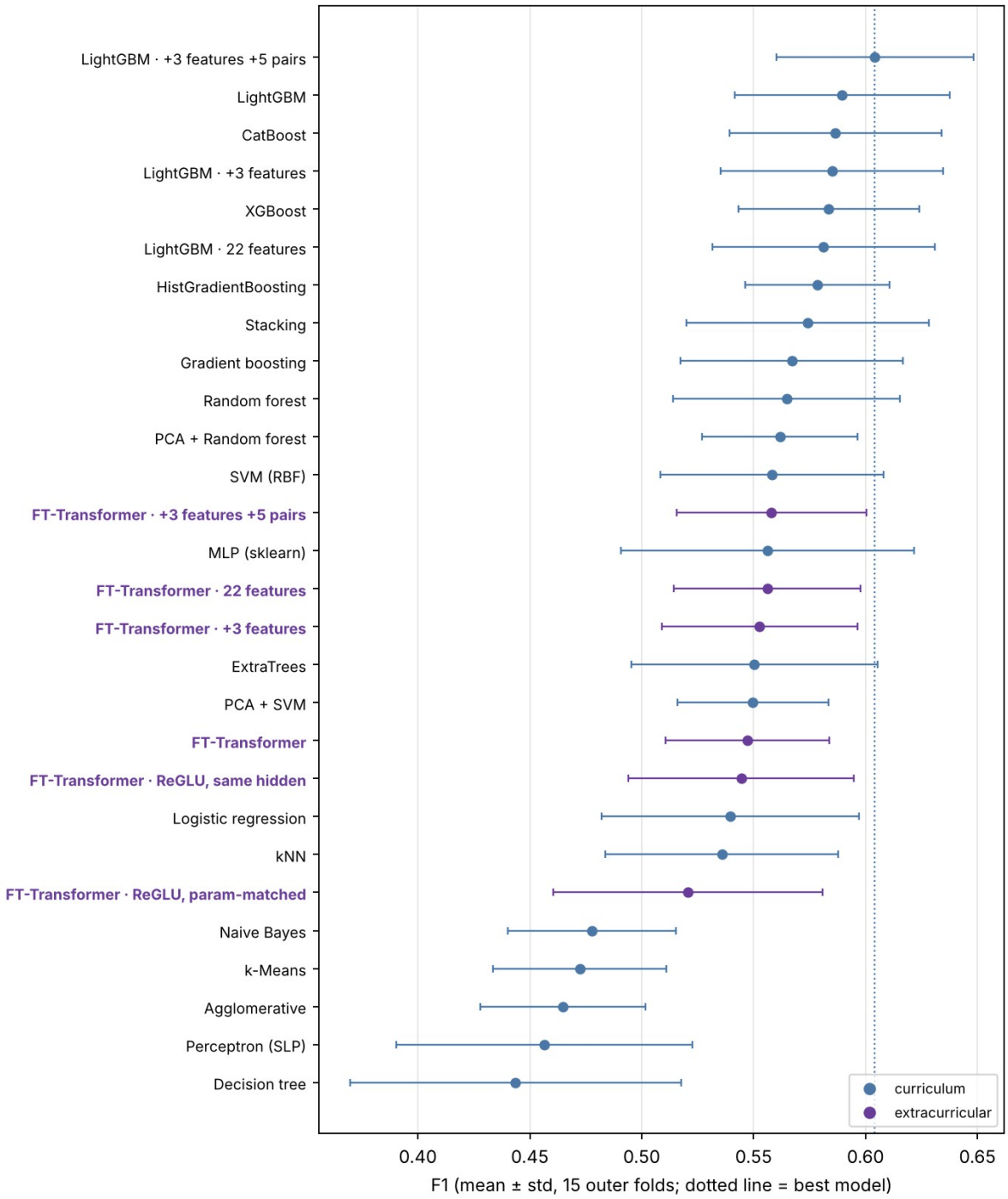
- สรุปคำตอบทั้งหมดในบทที่ 5 และต่อยอด: ปรับ hyper-parameter ของ FT-Transformer-lite และ NAM ด้วย Optuna ในงบเท่ากัน (หัวข้อ 6.4)

บทที่ **5** ผลการทดลองและการเปรียบเทียบ

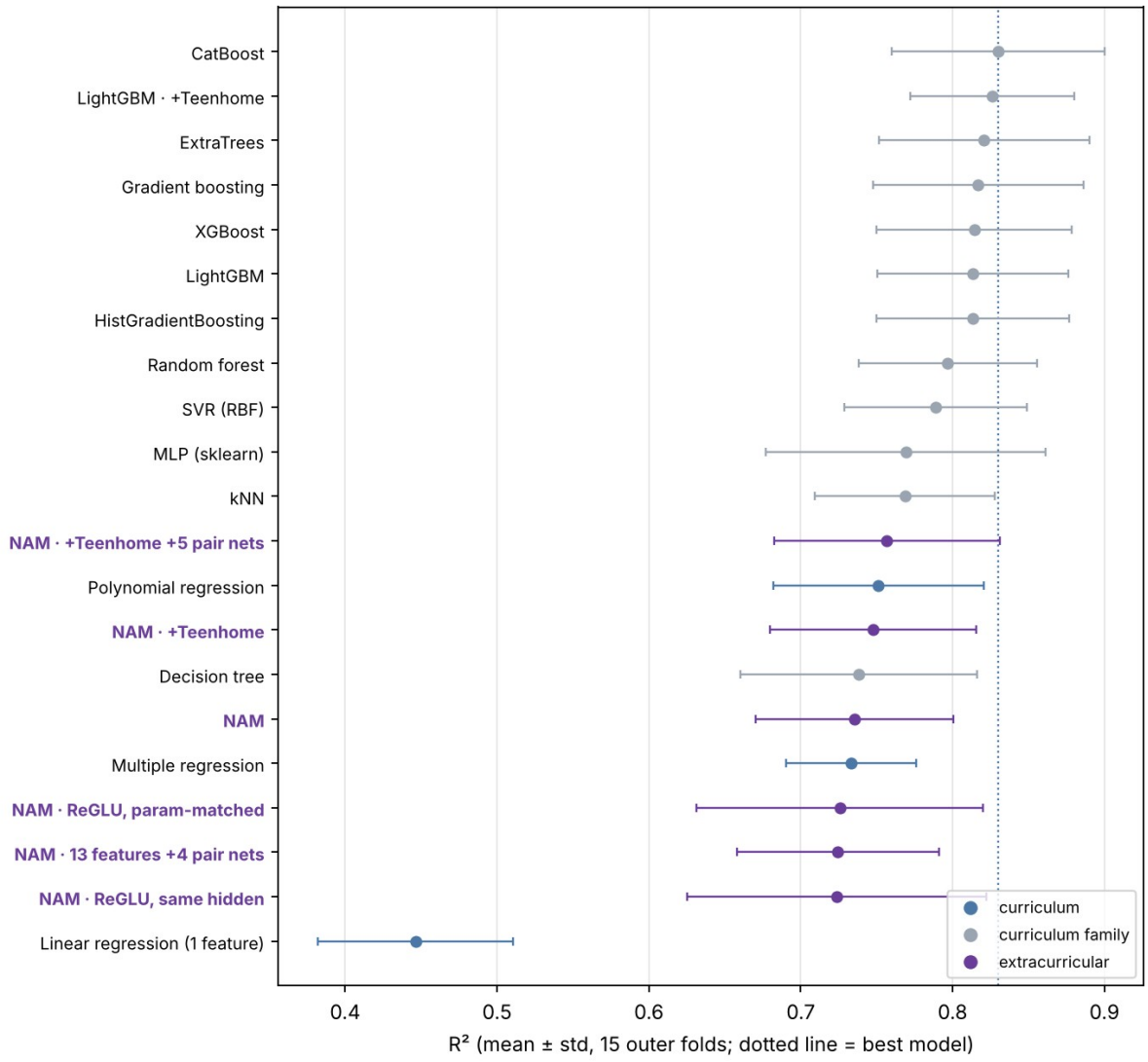
บทนี้ตอบคำถามข้อ 3 ของอาจารย์ก่อน: หัวข้อ 5.1–5.5 ใช้ผลรอบที่สอง (15 folds เดียวกันทุกโมเดล) ส่วนหัวข้อ 5.6–5.10 เก็บผลรอบแรก (V1.0–V3.1) ไว้เพื่อแสดงเส้นทางการพัฒนา

5.1 กระดานคะแนนทุกโมเดล **(15 folds** เดียวกัน**)**

บน 15 folds เดียวกัน extracurricular ที่ดีที่สุดได้ อันดับ **13** จาก **28** ใน classify และอันดับ **12** จาก **21** ใน regression อันดับบนสุดของทั้งสองงานเป็น tree boosting



ภาพที่ **5.1** อันดับ F1 ของทุกโมเดล (classify)



ภาพที่ **5.2** อันดับ R^2 ของทุกโมเดล (regression)

- classify: มีโมเดลในหลักสูตร 12 ตัวอยู่เหนือ FT-Transformer-lite และ 10 ตัวอยู่ใต้ (รวม Logistic regression, kNN, Naïve Bayes)
- classify: FT-Transformer-lite อยู่กลุ่มเดียวกับ SVM (0.558) และ MLP (0.556)
- regression: ทุกโมเดลเหนือ NAM คือโมเดล classify ในหลักสูตรที่นำมาใช้กับ regression (curriculum family)
- regression: NA^{2}M อยู่เหนือโมเดล regression ทั้งสามตัวในหลักสูตร (Polynomial, Multiple, Linear)

- ความแกว่งระหว่าง fold (SD 0.03–0.10) ใหญ่กว่าความต่างระหว่างโมเดลใกล้เคียงกัน การตัดสินจึงใช้การทดสอบแบบจับคู่ในหัวข้อ 5.2

5.2 extracurricular ชนะหรือไม่

ไม่ชนะ ทั้งสองงานแพ้ tree boosting ที่ดีที่สุดในหลักสูตร gap คือ 0.046 F1 และ 0.073 R^2 และ 95% CI ไม่ครอบศูนย์

คู่ที่เทียบ (curriculum – extracurricular)	คะแนน	**gap**	95% CI
classify: LightGBM + ฟีเจอร์คู่ – FT-Transformer-lite + ฟีเจอร์คู่ (ดีที่สุด vs ดีที่สุด)	0.604 vs 0.558	**0.046**	0.022 ถึง 0.070
classify: LightGBM – FT-Transformer-lite (รุ่นพื้นฐานทั้งคู่)	0.590 vs 0.547	**0.042**	0.025 ถึง 0.061
regression: CatBoost – NA²M (ดีที่สุด vs ดีที่สุด)	0.830 vs 0.757	**0.073**	0.046 ถึง 0.100
regression: CatBoost – NAM (รุ่นพื้นฐาน)	0.830 vs 0.735	**0.095**	0.065 ถึง 0.123
regression: Polynomial – NA²M (โมเดล regression ในหลักสูตร)	0.751 vs 0.757	**–0.006**	–0.030 ถึง 0.019
regression: Multiple – NA²M	0.733 vs 0.757	**–0.023**	–0.045 ถึง 0.003

ตารางที่ **5.1** ผลต่างระหว่างโมเดล curriculum และ extracurricular

- regression: gap กับ CatBoost ผ่านทุกการทดสอบ (Wilcoxon p = 0.0003, corrected t-test p = 0.034) และ CatBoost ชนะ 14 จาก 15 fold
- classify: Wilcoxon ยืนยัน (p = 0.003) แต่ corrected t-test ซึ่งเข้มกว่าได้ p = 0.11 ส่วน CI ทั้งช่วงอยู่เหนือศูนย์
- classify: FT-Transformer-lite เสมอกับ HistGradientBoosting (gap 0.020, CI –0.002 ถึง 0.043) และ Stacking (0.016, CI –0.009 ถึง 0.041)

- regression: NA²M เสมอกับ Polynomial และสูงกว่า Multiple เล็กน้อย (ชนะ 12 จาก 15 fold, Wilcoxon p = 0.035)
- คำตอบต่อคำถามข้อ 3 ของอาจารย์: extracurricular ไม่ได้ชนะจริง ตัวเลข V2.0 ที่เคยดูเหมือนชนะ (F1 0.604) มาจากการเลือก epoch บนชุดทดสอบ

5.3 ทำไม **extracurricular** แพ้

สาเหตุต่างกันตามงาน: classify แพ้เพราะ ข้อมูลน้อยและรูปแบบการเรียนผลรายฟีเจอร์ ส่วน regression แพ้เพราะ **NAM** จับ **interaction** ระหว่างฟีเจอร์ไม่ได้ เนื่องจาก NAM เรียนผลของแต่ละฟีเจอร์แยกกันแล้วนำมาบวก จึงไม่เห็นผลที่เกิดจากหลายฟีเจอร์ร่วมกัน ส่วนที่ขาดไปนี้คิดเป็น 55% ของ gap (0.044 จาก 0.078, หลักฐานจาก V7 หัวข้อ 4.10)

สาเหตุ	หลักฐาน	classify (FT-Transformer-lite)	regression (NAM)
1. รูปแบบการเรียนผลรายฟีเจอร์: ต้นไม้แบ่งค่าเป็นขั้นได้ทันที ส่วน neural network ต้องเรียนเป็นเส้นโค้งเรียบ	ต้นไม้ลึก 1 ชั้น (ไม่มี interaction) ชนะ	**+0.023** F1	**+0.035** R^2
2. interaction: NAM บวกผลแยกกัน ต้นไม้ผสมได้ทุกคู่	ลึก 1 → เต็ม	แทบไม่มีผล (+0.009)	**+0.043** R^2
3. ข้อมูลน้อย (ราว 1,800 แถวฝึก, ลูกค้าตอบรับราว 270)	learning curve	gap หด 0.068 → 0.034 เมื่อข้อมูล ×4	คงที่ราว 0.06
4. การปรับจูนไม่เท่ากัน	curriculum ผ่าน Optuna 30 trials ต่อ fold	hyper-parameter คงที่จาก V2.0	hyper-parameter คงที่จาก V2.0
5. ส่วนเสริมที่ลองแล้วไม่ช่วย	V4, V5	ReGLU –0.003, ฟีเจอร์คู่ +0.002	ReGLU –0.011, NA^2M +0.009

ตารางที่ **5.2** สาเหตุที่โมเดล extracurricular ให้ผลต่ำกว่า

- ข้อ 1–3 มีหลักฐานจากการทดลองโดยตรง ข้อ 4 ยังไม่ได้วัด จึงเป็นข้อจำกัด (หัวข้อ 5.12)
- ผลนี้ตรงกับงานวิจัยที่พบว่า tree boosting ยังเหนือกว่า deep learning บนตารางขนาดกลางและเล็ก เพราะ neural network เอนเอียงไปทางฟังก์ชันที่เรียบ [13]
- FT-Transformer ต้นฉบับ [1] ทดสอบบนชุดข้อมูลหลักหมื่นถึงหลักแสนแถว และใช้โมเดลใหญ่กว่าเวอร์ชัน lite ของเรา (หัวข้อ 4.4.1)

5.4 ทำไม gap ไม่ใหญ่

extracurricular ทำได้ **87% (classify)** และ **91% (regression)** ของส่วนที่โมเดลที่ดีที่สุดทำได้เหนือ baseline ง่าย ๆ เพราะสัญญาณส่วนใหญ่ในข้อมูลนี้เป็นแบบที่ทุกโมเดลดี ๆ เก็บได้

โมเดล	คะแนน (15 folds)	**%** ของส่วนเพิ่มของโมเดลที่ดีที่สุด	fold ที่ชนะโมเดลที่ดีที่สุด
classify: LightGBM (พื้นฐาน)	F1 0.590	**96%**	6 / 15
classify: FT-Transformer-lite + ฟีเจอร์คู่	F1 0.558	**87%**	3 / 15
classify: MLP	F1 0.556	**86%**	5 / 15
classify: Logistic regression	F1 0.540	**81%**	0 / 15
regression: LightGBM (พื้นฐาน)	R^2 0.814	**98%**	4 / 15
regression: NA^{2}M	R^2 0.757	**91%**	1 / 15
regression: Polynomial	R^2 0.751	**90%**	1 / 15
regression: Multiple	R^2 0.733	**88%**	1 / 15

ตารางที่ **5.3** สัดส่วนผลที่ได้เหนือ baseline เทียบกับโมเดลที่ดีที่สุด

- baseline: classify คือทำนายทุกคนว่า "ตอบรับ" (F1 = 0.260) ส่วน regression คือทำนายค่าเฉลี่ย (R^2 = 0)
- แม้แต่โมเดลเชิงเส้นอย่าง Logistic regression (classify) และ Multiple regression (regression) ก็ได้ 81% และ 88% แล้ว เพราะฟีเจอร์เด่น (เช่น `MntWines`, `Recency`, `AcceptedCmp5`) ส่งผลเกือบเป็นทิศทางเดียว (ค่ายิ่งมากผลยิ่งไปทางเดิม) โมเดลที่ซับซ้อนกว่าจึงเหลือส่วนให้ทำคะแนนเพิ่มได้ไม่มาก
- ทุกโมเดลใช้ฟีเจอร์เดียวกันและอ่านสัญญาณหลักตัวเดียวกัน (attention ของ FT-Transformer-lite และ gain ของ LightGBM ชี้ `Recency`, `Customer_Tenure_Days` เหมือนกัน)

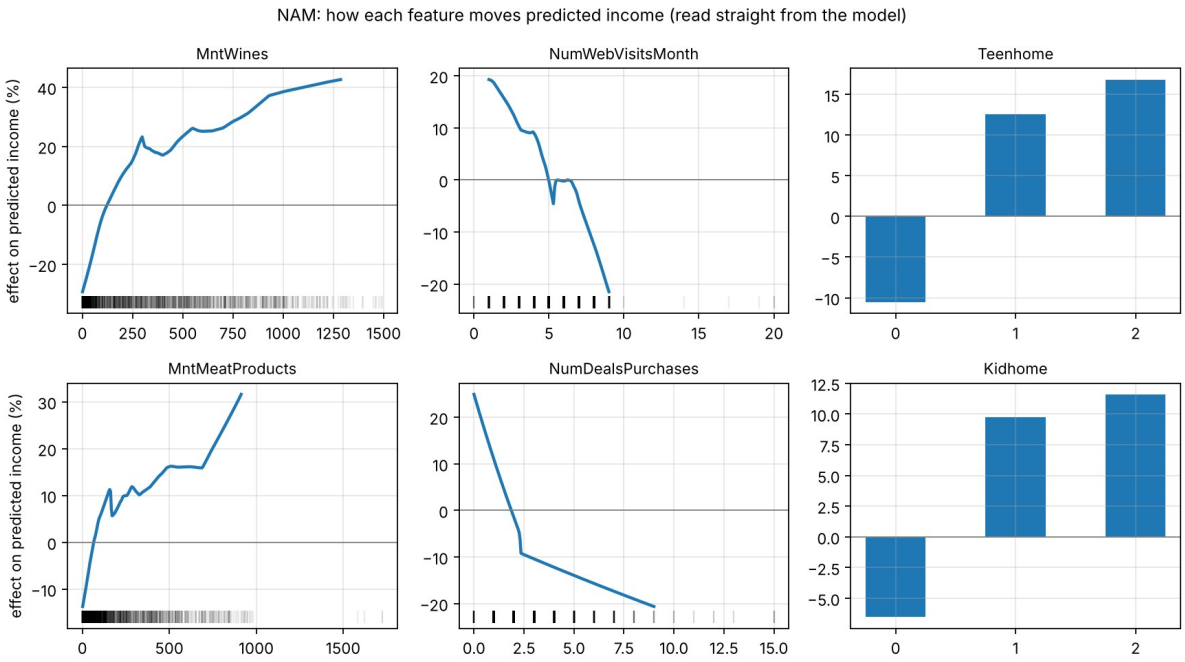
- ความแกว่งระหว่าง fold (SD 0.04–0.07) ใกล้เคียงกับ gap FT-Transformer-lite จึงชนะ LightGBM ตัวดีที่สุดได้ 3 จาก 15 fold
- ผลคือ "แพ้จริงแต่แพ้ไม่มาก": อยู่กลางตาราง และเหนือโมเดลง่ายทั้งหมดในหลักสูตร

5.5 ข้อได้เปรียบที่โมเดลอื่นไม่มี

ข้อได้เปรียบของ extracurricular ไม่ใช่ความแม่นยำ แต่คือ **NAM** อธิบายได้ทั้งโมเดล ทุกคำทำนายคือผลรวมของผลจากแต่ละฟีเจอร์ จึงดูได้ตรง ๆ ว่าแต่ละฟีเจอร์เพิ่มหรือลดรายได้ที่ทำนายเท่าไร โดยไม่ต้องใช้เครื่องมือประมาณค่าเพิ่ม และยังแม่นกว่าหรือเท่ากับโมเดล regression ในหลักสูตร โดยแลกกับ R^2 ที่ต่ำกว่า CatBoost 0.073

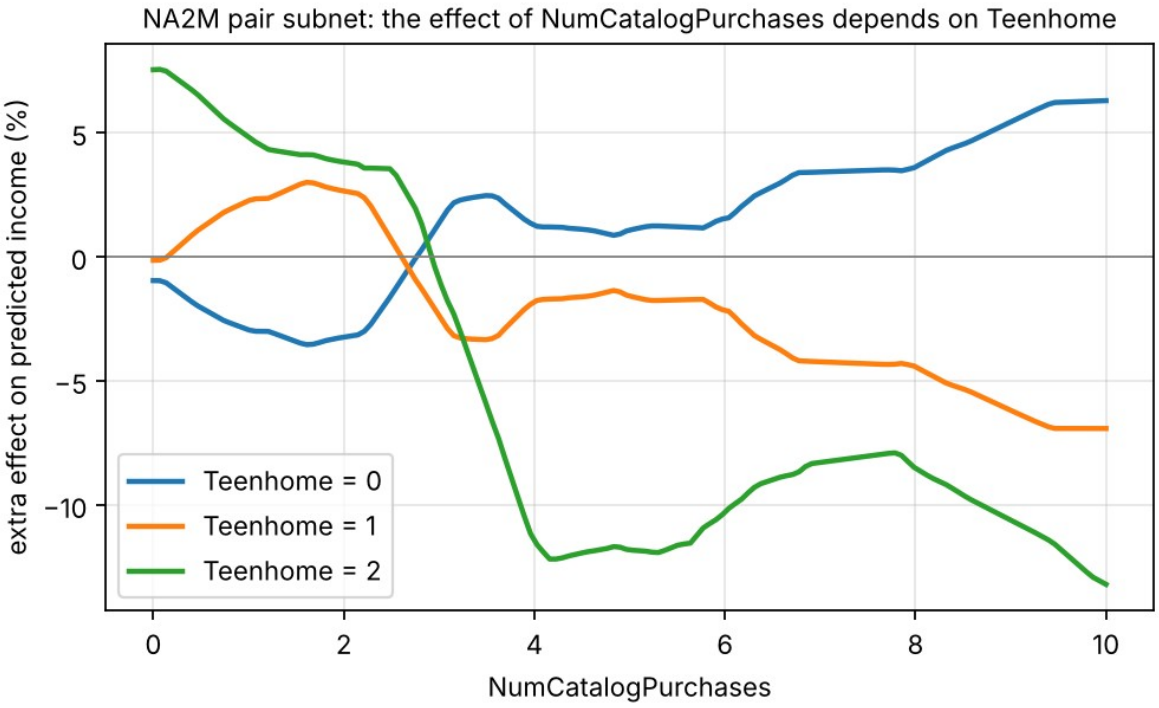
คุณสมบัติ (regression, 15 folds)	NA^2M (extracurricular)	Polynomial / Multiple (หลักสูตร)	CatBoost (ดีที่สุด)
R^2	**0.757**	**0.751 / 0.733**	**0.830**
MAE ($)	6,561	6,957 / 7,585	5,456
อ่านผลต่อฟีเจอร์	กราฟ 1 เส้นต่อฟีเจอร์ (14 เส้น) + กราฟคู่ 5 ภาพ	104 สัมประสิทธิ์ (degree 2) / เส้นตรง 13 เส้น	ต้องประมาณภายหลัง (เช่น SHAP)
อธิบายรายลูกค้า	ผลรวมของชิ้นส่วน = คำทำนายพอดี	ได้ แต่ต้องรวมผลคูณไขว้	ได้เพียงค่าประมาณ
เห็น interaction	เป็นกราฟคู่ตรง ๆ	ซ่อนอยู่ในผลคูณ 78 ตัว / ไม่มี	มีแต่มองไม่เห็น

ตารางที่ **5.4** เปรียบเทียบ NA^2M กับโมเดล regression อื่น



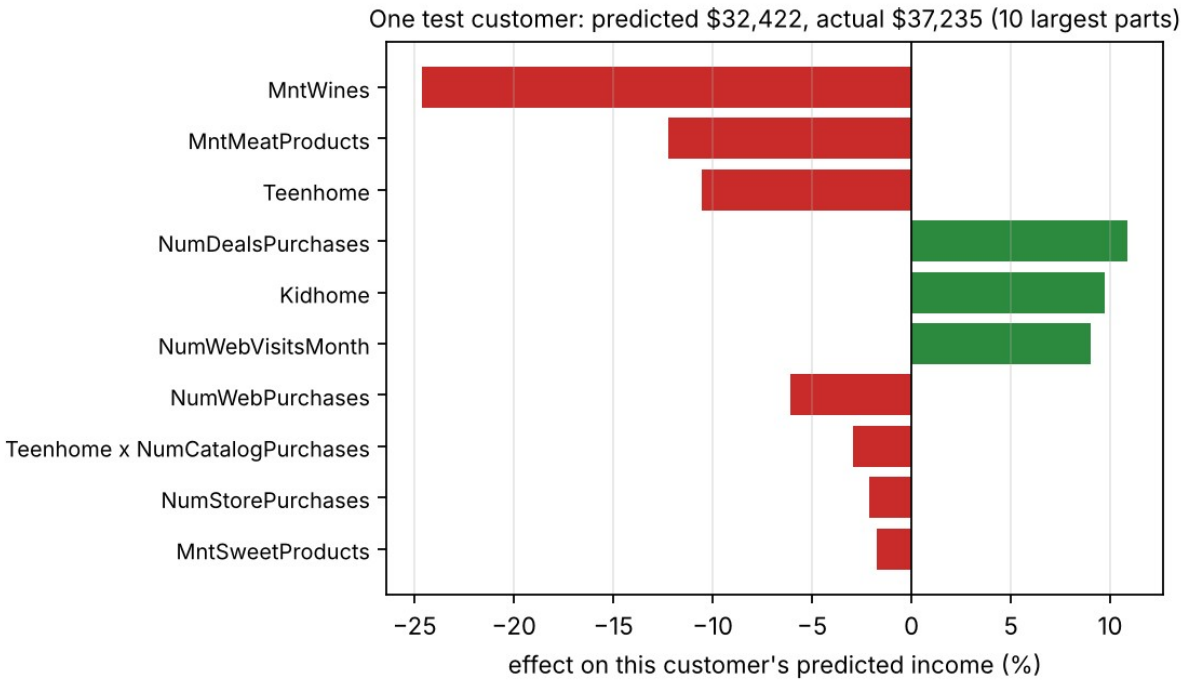
ภาพที่ **5.3** shape function ของ NA²M

- MntWines สูงขึ้น → รายได้ทำนายสูงขึ้นถึง +43% ที่ 1,300 ดอลลาร์ แต่ชันที่สุดช่วง 0–300
- NumWebVisitsMonth และ NumDealsPurchases เป็นเส้นลดลง: เข้าเว็บบ่อยและซื้อของลดราคาบ่อย → รายได้ทำนายต่ำลงราว 20%
- Teenhome และ Kidhome: เมื่อยอดซื้อเท่ากัน บ้านที่มีเด็กถูกทำนายรายได้สูงกว่า (เช่น Teenhome = 0 ทำให้ −10% แต่ = 2 ทำให้ +17%)
- กราฟเหล่านี้อ่านจากโมเดลโดยตรง ไม่ใช่การประมาณภายหลัง แต่เป็นความสัมพันธ์ที่โมเดลเรียนรู้ ไม่ใช่เหตุและผล



ภาพที่ **5.4** ผลร่วมของ NumCatalogPurchases และ Teenhome

- บ้านที่ไม่มีวัยรุ่น: ซื้อผ่านแคตตาล็อกมาก → รายได้เพิ่มอีก +6%
- บ้านที่มีวัยรุ่น 2 คน: ซื้อผ่านแคตตาล็อกมาก → รายได้ลดลงอีก −12%
- นี่คือ interaction ที่ EDA (หัวข้อ 2.9) พบเป็นอันดับ 1 และตอบคำถามข้อ 1 ของอาจารย์เป็นภาพ
- pair subnet ถูก purify ให้เหลือเฉพาะผลร่วม ส่วนเฉลี่ยย้ายไปอยู่ใน shape function [21] ผลรวมทั้งหมดไม่เปลี่ยน



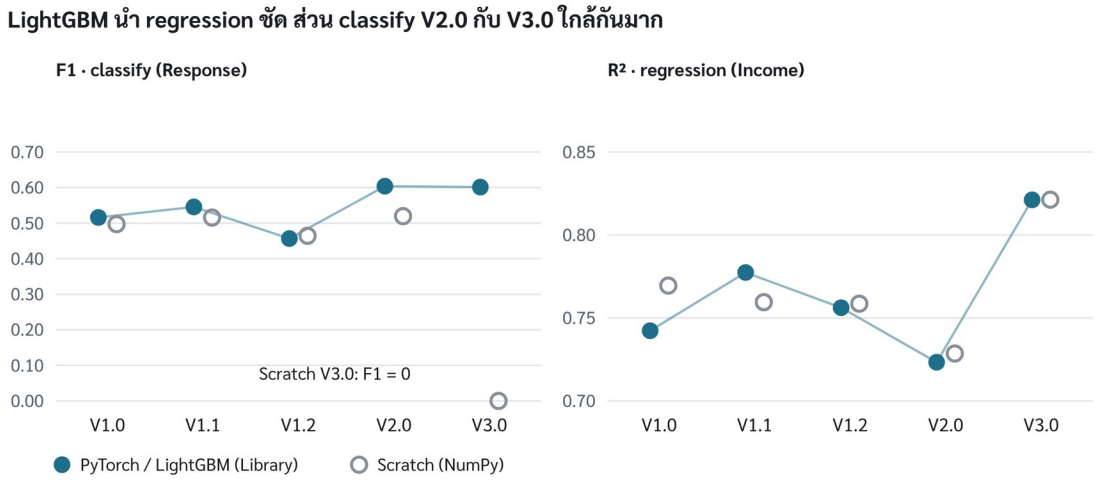
ภาพที่ **5.5** ตัวอย่างการอธิบายผลการทำนายรายลูกค้า

- ทำนาย $32,422 (จริง $37,235): `MntWines` ต่ำดึงลง −25% ส่วน `NumDealsPurchases`, `Kidhome`, `NumWebVisitsMonth` ดันขึ้นตัวละราว +10%
- ผลรวมของชิ้นส่วนทุกชิ้นเท่ากับคำทำนายของโมเดล (ต่างไม่เกิน 5×10^{-7}) จึงใช้อธิบายการตัดสินใจรายบุคคลได้
- CatBoost แม่นกว่า แต่คำอธิบายแบบเดียวกันต้องประมาณจากต้นไม้หลายร้อยต้น

FT-Transformer-lite (classify)

- ไม่มีข้อได้เปรียบด้านความแม่นยำ: เสมอกับ MLP และ SVM และสูงกว่า Logistic regression เล็กน้อย (0.018, ไม่มีนัยสำคัญ)
- สิ่งที่ได้เพิ่มคือ attention ต่อลูกค้า (หัวข้อ 4.4) ซึ่งชี้ไปที่ฟีเจอร์เดียวกับ RF และ LightGBM (`Recency`, `Customer_Tenure_Days`) แต่ attention ไม่ใช่คำอธิบายที่ตรงตัวเหมือน NAM

5.6 ผลรอบแรก: กระดานคะแนน V1.0–V3.0 (split เดียว)



ภาพที่ **5.6** คะแนนของ V1.0–V3.0 บน split เดียว

เวอร์ชัน	PyTorch / LightGBM	Scratch	ส่วนต่าง
V1.0 MLP ตื้น	0.5161	0.4974	0.0187
V1.1 MLP หลาย Layer	0.5455	0.5158	0.0297
V1.2 ตัดฟีเจอร์ 3/3	0.4566	0.4645	0.0079
V2.0 FT-Transformer-lite	**0.6038**	0.5197	0.0841
V3.0 LightGBM	0.6014	0.0000	0.6014

ตารางที่ **5.5** F1 ของ V1.0–V3.0 (classify)

เวอร์ชัน	PyTorch / LightGBM	Scratch	ส่วนต่าง
V1.0 MLP ตื้น	0.7423	0.7695	0.0272
V1.1 MLP หลาย Layer	0.7774	0.7595	0.0179
V1.2 ตัดฟีเจอร์ 3/3	0.7562	0.7586	0.0024
V2.0 NAM	0.7233	0.7286	0.0053
V3.0 LightGBM	**0.8212**	**0.8213**	0.0001

ตารางที่ **5.6** R^2 ของ V1.0–V3.0 (regression)

เวอร์ชัน	PyTorch / LightGBM	Scratch	ส่วนต่าง
V1.0 MLP ตื้น	11,064.7	10,463.3	601.4
V1.1 MLP หลาย Layer	10,283.8	10,688.1	404.3
V1.2 ตัดฟีเจอร์ 3/3	10,761.2	10,708.1	53.1
V2.0 NAM	11,463.5	11,353.3	110.2
V3.0 LightGBM	**9,216.7**	**9,212.9**	3.8

ตารางที่ **5.7** RMSE ของ V1.0–V3.0 (regression)

- Baseline แบบง่าย: ทำนายทุกแถวว่า “ไม่ตอบรับ” ได้ accuracy 0.8504 (381 จาก 448 แถว) แต่ F1 = 0 จึงใช้ F1 เป็นตัวตัดสินหลักทั้งบท
- ทุกโมเดลในตารางทำนายเก่งกว่าการทำนายค่าเฉลี่ย ($R^2 = 0$) อย่างชัดเจน แต่ช่วง R^2 0.72–0.82 ชี้ว่าความแตกต่างอยู่ที่ชนิดโมเดล

- classify F1 สูงสุดคือ V2.0 (0.6038) แต่ใกล้ V3.0 (0.6014) มาก จะตัดสินได้ควรดูผล 5-fold ในหัวข้อ 5.8 และ 15 folds ในหัวข้อ 5.1

5.7 ผลรอบแรก**:** เทียบกับเวอร์ชันก่อนหน้า

การเปลี่ยน	Δ F1 (Library)	Δ R^2 (Library)
V1.0 → V1.1 (หลาย Layer + ResNet)	+0.0294	+0.0351
V1.1 → V1.2 (ตัดชั้น 2/3)	–0.0889	–0.0212
V1.1 → V2.0 (FT-Transformer-lite / NAM)	+0.0583	–0.0541
V1.1 → V3.0 (LightGBM)	+0.0559	+0.0438

ตารางที่ **5.8** ผลต่างจากเวอร์ชันก่อนหน้า (Library)

- การปรับที่ได้ผลทั้งสองงานมีสองจุด: V1.0 → V1.1 และ V1.1 → V3.0
- V1.2 แย่ลงทั้งสองงาน ยืนยันว่าฟีเจอร์ชั้น 2/3 มีสัญญาณจริง (สอดคล้องกับหัวข้อ 2.7 ที่ให้พิจารณาชั้นนี้รายตัว)
- V2.0 สลับทิศ: architecture ใหม่ช่วย classify แต่ทำให้ regression แย่ลง (NAM ยอมแลกความแม่นยำกับความอธิบายได้)
- V3.0 ใน classify ไม่แซง V2.0 (–0.0024) แต่สูงกว่า V1.1 ชัดเจน

5.8 ผลรอบแรก**: 5-fold (V1.3, V2.1, V3.1)**

โมเดล	F1 (mean ± SD)	R^2 (mean ± SD)
V1.3 MLP หลาย Layer	0.5502 ± 0.0200	0.7553 ± 0.0463
V2.1 FT-Transformer-lite / NAM	0.5723 ± 0.0380	0.7395 ± 0.0375
V3.1 LightGBM	**0.5944 ± 0.0447**	**0.8260 ± 0.0427**

ตารางที่ **5.9** ผล 5-fold ของรอบแรก

(ค่าของฝั่ง PyTorch / LightGBM เท่านั้น ผล Scratch อยู่ในหัวข้อ 4.6)

โมเดล · ตัววัด	split เดียว	5-fold mean	ส่วนต่าง
MLP หลาย Layer · F1	0.5455	0.5502	–0.0047
FT-Transformer-lite · F1	0.6038	0.5723	+0.0315
LightGBM · F1	0.6014	0.5944	+0.0070
MLP หลาย Layer · R^2	0.7774	0.7553	+0.0221
NAM · R^2	0.7233	0.7395	–0.0162
LightGBM · R^2	0.8212	0.8260	–0.0048

ตารางที่ **5.10** เปรียบเทียบผล split เดียวกับค่าเฉลี่ย 5-fold

- ทุกคู่ไม่เกิน ±0.025 ยกเว้น F1 ของ V2.0 ซึ่งเลือก epoch บนชุดทดสอบ จุดที่ “ดีเกินจริงที่สุด” คือ F1 ของ V2.0 (+0.032)
- regression: LightGBM นำอันดับในทุก fold เทียบกับ MLP หลาย Layer แม้ผลต่างของค่าเฉลี่ย (+0.071) จะใกล้ SD ของ fold (0.04–0.05) ก็ตาม เพราะ fold เดียวกันทุกโมเดลยากพอกัน การเทียบทีละคู่จึงสะอาดกว่าการเทียบ SD รวม
- classify: สองอันดับบน (V3.1 กับ V2.1) ต่างกัน 0.022 ซึ่งเล็กกว่า SD จึงสรุปจาก 5 fold ได้แค่ว่าทั้งสองแซง MLP หลาย Layer (FT-Transformer-lite 4 จาก 5 fold, LightGBM 5 จาก 5 fold) ผล 15 folds ที่แยกได้ชัดกว่าอยู่ในหัวข้อ 5.1–5.2

5.9 Parity: Library กับ Scratch

เวอร์ชัน (split เดียว)	Δ F1 classify	ผลตามเกณฑ์ 0.05	Δ R^2 regression
V1.0	0.0187	ผ่าน	0.0272
V1.1	0.0297	ผ่าน	0.0179
V1.2	0.0079	ผ่าน	0.0024
V2.0	0.0841	ไม่ผ่าน	0.0053
V3.0	0.6014	ไม่ผ่าน	0.0001

ตารางที่ **5.11** ผล Parity Check บน split เดียว

เวอร์ชัน (5-fold)	ส่วนต่างค่าเฉลี่ย F1	ส่วนต่างค่าเฉลี่ย R^2
V1.3	0.0062	0.0020
V2.1	0.0003	0.0044
V3.1	0.0173	0.0026

ตารางที่ **5.12** ผล Parity Check บน 5-fold

- parity ไม่ผ่านเกิดใน classify split เดียวสองครั้ง (V2.0, V3.0) และ regression parity แนบในทุกเวอร์ชัน
- ทั้งสองครั้งมาจากการเลือกจุดหยุดฝึก (เลือก epoch ที่ไม่เท่ากันใน V2.0 · early stopping บน BCE หยุดที่รอบ 8 ใน V3.0) ไม่ใช่สูตรคณิตศาสตร์: V3 ผ่าน sanity check
- ผล 5-fold ยืนยัน: ส่วนต่างเฉลี่ยทุกคู่อยู่ในเกณฑ์ (≤0.018) และ scratch ไม่เกิด F1 = 0 ใน fold ใดเลย
- ผล regression (Δ R^2 ไม่เกิน 0.03 ทุกครั้ง) ทำให้เชื่อได้ว่า neural network และ GBDT ที่เขียนเองด้วย NumPy ทำงานเทียบเท่าไลบรารีอ้างอิง

5.10 ฟีเจอร์ที่หลายวิธีชี้ตรงกัน

- classify: Recency และ Customer_Tenure_Days อยู่อันดับบนทั้งใน RF (หัวข้อ 2.6) gain ของ LightGBM และ attention ของ FT-Transformer-lite ตามด้วย MntMeatProducts, Income, MntGoldProds
- regression: MntWines และ MntMeatProducts นำ ตามด้วย NumWebVisitsMonth, NumCatalogPurchases, NumDealsPurchases ซึ่งเป็นพฤติกรรมการซื้อทั้งหมด
- ความหมายทางธุรกิจ: ฟีเจอร์ที่ขึ้นอันดับบนคือ Recency, Customer_Tenure_Days และ MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds แต่การจัดอันดับความสำคัญไม่บอกทิศทางของผล จึงยังไม่สรุปว่าฟีเจอร์เหล่านี้เพิ่มหรือลดโอกาสตอบรับ

5.11 อภิปราย

- ภาพรวม**:** tree boosting ดีที่สุดทั้งสองงาน (F1 0.590–0.604, R^2 0.814–0.830) สอดคล้องกับงานวิจัยบนข้อมูลตาราง [13]
- **extracurricular** แพ้จริงแต่ไม่มาก**:** อยู่กลางตารางทั้งสองงาน (หัวข้อ 5.2–5.4)
- บทเรียนเรื่องวิธีประเมิน**:** FT-Transformer-lite เคยดูเหมือนชนะ (F1 0.604) เพราะเลือก epoch บนชุดทดสอบ เมื่อเลือกบน inner fold คะแนนเหลือ 0.547
- **ReGLU:** ช่วย Transformer ขนาดใหญ่ แต่ไม่ช่วยบนข้อมูล 2,200 แถว (หัวข้อ 4.7)
- **Interaction** มีจริง (หัวข้อ 2.9) แต่ช่วยเฉพาะโมเดลที่ผสมฟีเจอร์เองไม่ได้หรือผสมยาก (LightGBM, NAM) ไม่ช่วย FT-Transformer-lite ที่มี attention อยู่แล้ว
- ความอธิบายได้มีราคา**:** NAM แลก R^2 0.073 จาก CatBoost กับคำอธิบายที่ตรงตัว และยังเสมอหรือดีกว่าโมเดล regression ในหลักสูตร
- **classify** มีเพดาน**:** ทุกโมเดลได้ F1 $\leq$ 0.60 ฟีเจอร์ที่มีให้สัญญาณจำกัด การเปลี่ยนโมเดลเพียงอย่างเดียวจึงเพิ่มได้ไม่มาก

5.12 ข้อจำกัดของการทดลอง

- FT-Transformer-lite และ NAM ไม่ได้ผ่าน Optuna (ใช้ hyper-parameter จาก V2.0) ขณะที่โมเดลในหลักสูตรผ่าน 30 trials ต่อ fold การเทียบจึงเอื้อฝั่ง curriculum อยู่บ้าง
- inner fold ใช้ 3 แทน 5 ตามแผนเดิม เพราะเวลาคำนวณ
- คู่ interaction ใน V5 เลือกจาก EDA ที่ใช้ทุกแถว ผลบวกของ V5 จึงอาจสูงเกินจริงเล็กน้อย
- การเลือกตัวดีที่สุดจากหลายรุ่นบน fold เดียวกันทำให้คะแนนสูงสุดของแต่ละกลุ่มดูดีกว่าจริงเล็กน้อย (เกิดทั้งสองฝั่ง)
- learning curve ใช้ 5 folds ของ seed เดียว จึงเป็นหลักฐานแนวโน้ม ไม่ใช่ตัวเลขแม่นยำ
- รอบแรก (V1.1, V1.2, V2.0, V2.1) เลือก checkpoint บนข้อมูลที่ใช้รายงาน และใช้ threshold 0.5 คงที่ รอบสองแก้แล้ว ตัวเลขรอบแรกจึงใช้ดูเส้นทางการพัฒนาเท่านั้น
- Scratch รอบสองใช้จำนวน epoch และ threshold จาก PyTorch ส่วนความถูกต้องตรวจด้วยน้ำหนักเดียวกัน (หัวข้อ 4.7)
- การวิเคราะห์ EDA (หัวข้อ 2.3–2.9) คำนวณก่อนลบแถว `Income` $666,666 จึงใช้ 2,237 / 2,213 แถว ไม่ใช่ 2,236 / 2,212 แถวของการเทรน
- Scratch GBDT สร้างต้นไม้แบบ depth-wise ในขณะที่ LightGBM เป็น leaf-wise จึงไม่ใช่สำเนาเหมือนทุกจุด

บทที่ 6 สรุป ข้อจำกัด และแนวทางต่อยอด

6.1 สรุปผล

- ข้อสรุปหลัก: `Income` ทำนายได้ดี (R^2 สูงสุด 0.830) ส่วน `Response` มีเพดาน (F1 ≤ 0.604) เพราะฟีเจอร์ที่มีให้สัญญาณจำกัด
- คำถามที่ **1 — Feature interaction:** มีคู่ที่ส่งผลร่วมกันจริง (33 จาก 296 คู่ใน classify, 156 คู่ใน regression) ส่วนใหญ่เกี่ยวกับ `Teenhome` และ `NumDealsPurchases` ที่เคยถูกตัด เมื่อใส่เข้าโมเดล ช่วย LightGBM (+0.023 F1) และ NAM (+0.021 R^2) แต่ไม่ช่วย FT-Transformer-lite

- คำถามที่ **2 — ReGLU:** FT-Transformer ใช้เพราะ gate ช่วย Transformer ที่ฝึกบนข้อมูลมาก บนข้อมูลของเราไม่ช่วยทั้ง FT-Transformer-lite (–0.003 ถึง –0.027 F1) และ NAM (–0.010 ถึง –0.011 R^2) และทำให้ผลแกว่งขึ้น
- คำถามที่ **3 — extracurricular** ชนะจริงหรือไม่**:** ไม่ชนะ คะแนนห่างจากโมเดลที่ดีที่สุด 0.046 F1 (classify) และ 0.073 R^2 (regression)
- สาเหตุที่แพ้**:** classify — รูปแบบการเรียนผลรายฟีเจอร์และข้อมูลน้อย (gap หดครึ่งหนึ่งเมื่อข้อมูล ×4) · regression — NAM จับ interaction ระหว่างฟีเจอร์ไม่ได้ (คิดเป็น 55% ของ gap)
- ข้อได้เปรียบที่เหลือ**:** NA^2M เสมอหรือดีกว่าโมเดล regression ในหลักสูตร (R^2 0.757 เทียบ 0.751 / 0.733, MAE ต่ำกว่า $396–$1,024) และอธิบายทุกคำทำนายได้ตรงตัว รวมถึงภาพ interaction
- การคำนวณจากศูนย์ด้วย **NumPy:** รอบสองตรงกับ PyTorch ทุกโมเดลบนน้ำหนักเดียวกัน (ต่างไม่เกิน 10^{-13}) รอบแรกตรงใน 8 จาก 10 กรณีของ split เดียว (หัวข้อ 5.9)

6.2 ข้อเสนอการใช้งาน

- ต้องการความแม่นยำสูงสุด: LightGBM + ฟีเจอร์คู่ (classify, F1 0.604) และ CatBoost (regression, R^2 0.830)
- ต้องอธิบายผลรายลูกค้าหรือให้ฝ่ายธุรกิจตรวจได้: NA^2M แลกกับ R^2 ที่ต่ำกว่า CatBoost 0.073
- classify: ใช้คะแนนจัดอันดับลูกค้า และตั้ง threshold ตามต้นทุนแคมเปญ ไม่ใช้ค่า 0.5 คงที่

6.3 ข้อจำกัด

ข้อจำกัดทั้งหมดอยู่ในหัวข้อ 5.12 ข้อที่กระทบข้อสรุปมากที่สุดคือ FT-Transformer-lite และ NAM ยังไม่ได้ปรับ hyper-parameter ด้วยงบเดียวกับโมเดลในหลักสูตร และข้อมูลมีเพียงราว 2,200 แถว (ลูกค้าที่ตอบรับราว 330 แถว)

6.4 แนวทางต่อยอด

- ปรับ hyper-parameter ของ FT-Transformer-lite และ NAM ด้วย Optuna ในงบเท่ากับโมเดลในหลักสูตร (30 trials) เพื่อปิดข้อสงสัยเรื่องความเป็นธรรม

- ลอง extracurricular ที่ออกแบบมาสำหรับตารางเล็ก เช่น TabPFN [23] หรือ EBM (GA²M [17])
- เก็บข้อมูลเพิ่มสำหรับ classify เพราะ learning curve ชี้ว่า gap หดลงเมื่อข้อมูลมากขึ้น
- เลือกคู่ interaction ภายในแต่ละ fold เพื่อตัดผลบวกที่เกินจริงใน V5
- รายงาน confusion matrix และเส้น precision–recall พร้อม threshold ตามต้นทุนแคมเปญ
- รัน EDA ซ้ำบนแถวหลังตัดค่าผิดปกติ (2,236 / 2,212 แถว) ให้ตรงกับข้อมูลที่ใช้เทรน